Gisma: **Giant-Step-Small-Step Indexing for Approximate Similarity Search in Graph Databases**

Yikai Gong
Hong Kong Baptist University
ykgong@comp.hkbu.edu.hk

Lyu Xu
Nanyang Technological University
lyu.xu@ntu.edu.sg

Yun Peng
Guangzhou University
yunpeng@gzhu.edu.cn

Byron Choi
Hong Kong Baptist University
choi@hkbu.edu.hk

Jianliang Xu
Hong Kong Baptist University
xujl@comp.hkbu.edu.hk

Xin Huang
Hong Kong Baptist University
xinhuang@comp.hkbu.edu.hk

ABSTRACT

Approximate similarity search in graph databases retrieves graphs whose *graph edit distance* (GED) to a given query graph is within a given threshold. As the GED computation is NP-hard, indexing has been proposed to accelerate the query by reducing the number of such computations. As a fundamental property for indexing metric spaces, the concept of *doubling metrics* has been proposed but not investigated in the context of graph similarity search in the GED space. In this work, we take the first step to explore doubling metrics in the GED space. That is, the GED space is *doubling* — any ball of radius r can be covered by a bounded number (the doubling constant) of balls of radius $r/2$. Moreover, we reveal a two-stage phenomenon in this doubling space: when GED exceeds a threshold α, the doubling constant is small and the search on doubling-based indexes can efficiently locate the subspaces containing answer graphs; when GED is below α, small GED leads to more common GED computations during the query processing of the graphs in subspaces. Based on these insights, we propose **Gi**ant-Step-**Sma**ll-Step (Gisma), a novel two-stage index for approximate graph similarity search. The first stage of Gisma, namely `NetDAG`, is a doubling-based index to facilitate a coarse-grained search for answer subspaces with the lowest known time complexity, while Gisma's second stage `EPF` indexes for each subspace the common parts of GED computations, achieving an efficient fine-grained search for answer graphs with high recalls. Experiments on four benchmark datasets demonstrate that Gisma outperforms state-of-the-art solutions in both efficiency and effectiveness.

PVLDB Reference Format:
Yikai Gong, Lyu Xu, Yun Peng, Byron Choi, Jianliang Xu, and Xin Huang. Gisma: Giant-Step-Small-Step Indexing for Approximate Similarity Search in Graph Databases. PVLDB, 19(1): XXX-YYY, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/McKayGONG/Gisma.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

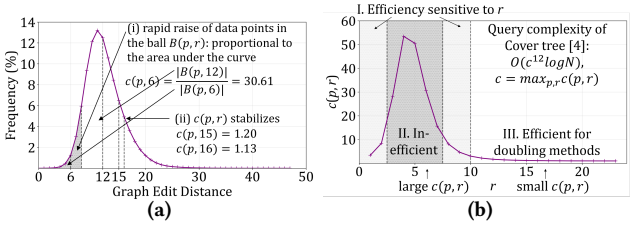


Figure 1: (a) Distribution of pairwise GEDs in the AIDS dataset (graphs with at most 15 vertices). The area under the curve up to r is proportional to $|B(p,r)|$, and the ratio $|B(p,2r)|/|B(p,r)|$ is $c(p,r)$; (b) $c(p,r)$ under the same setting. Three cases of doubling methods' efficiency: I. Efficiency sensitive to r, II. Inefficient, and III. Efficient.

1 INTRODUCTION

Similarity search in graph databases aims to retrieve graphs that are similar to a given query graph. It is a fundamental problem with a wide range of applications, including bioinformatics, cheminformatics, software engineering, and computer vision [1, 5, 11, 13, 16, 33, 36, 39, 40, 46, 47]. For example, in bioinformatics, molecular structures can be naturally represented as graphs, and a typical task is to search a compound database for molecules whose structures are similar to that of a given drug molecule.

For graph similarity search, the *graph edit distance* (GED), which is the minimum number of edit operations to transform a graph into another, has been widely used as the similarity metric in the literature [2, 6, 7, 27, 28, 34, 36, 43]. Specifically, given a query graph Q and a distance threshold τ, the GED similarity search is to retrieve all graphs from the graph database whose GED to Q is within τ. As GED computation/verification is NP-hard [44], the GED similarity search over large graph databases is computationally expensive, and hence, a variety of approximate methods have been proposed to efficiently answer graph similarity search queries in recent years.

Existing solutions. In general, the main ideas of existing works on approximate graph similarity search fall into two categories: (i) to reduce the number of GED computations, and (ii) to accelerate individual GED computations. Especially, for category (i), GHashing [36] uses graph neural networks with hashing for efficient filtering, while LAN [34] is a proximity graph-based approximate k-nearest neighbor search method on graph databases; for category (ii), App-BMao [41] extends the state-of-the-art exact GED verification method AStar-BMao [7] by using an iteration limit to compute approximate GED, achieving speedups with a slight accuracy loss,

whereas GEDHOT [8] adopts an ensemble strategy that combines supervised and unsupervised models to compute approximate GED.

There have also been works that study the dimensionality of the metric space for building efficient similarity search indexes [4, 9, 10, 19, 22, 26, 30], among which the doubling metrics measure the dimensionality through a doubling constant λ [30] and an expansion constant c [26]. In this work, we investigate doubling metrics to efficiently answer graph similarity search queries.

Doubling metrics. Given a set $\mathcal{X}$ of data points and a distance metric d, the metric space $(\mathcal{X}, d)$ is *doubling* if it has a finite *doubling constant* λ, defined as the smallest number such that: for every data point $p \in \mathcal{X}$ and every distance $r > 0$, the ball $B(p, 2r)$ can *always* be covered by at most λ balls of radius r, where a ball $B(p, r) = \{q \in \mathcal{X} \mid d(p, q) \leq r\}$ is defined as the set of points whose distances to the center point p are not larger than the radius r. For ease of presentation, we call such a metric space a *doubling space*, and denote by $\lambda(p, r)$ the minimum number of balls of radius r to cover $B(p, 2r)$. As another measure of dimensionality, the *expansion constant* c is defined as the maximum ratio $|B(p, 2r)|/|B(p, r)|$ over all p and r. We also denote the ratio $|B(p, 2r)|/|B(p, r)|$ for given p and r by $c(p, r)$. Although doubling metrics have been adopted to accelerate the approximate nearest neighbor search (ANNS) in some metric spaces, to the best of our knowledge, *doubling metrics have not been explored in the context of graph similarity search.*

Our proposal. In this work, we take the first step to investigate doubling metrics in the GED space to design an efficient index for graph similarity search. According to existing works [4, 22], the doubling constant λ and the expansion constant c are crucial to the time complexity of index-based search methods. Hence, we conduct a numerical evaluation of the distributions of GED, $\lambda(p, r)$, and $c(p, r)$.

• **Observations.** Figure 1(a) shows the distribution of exact GED between graphs in the AIDS dataset with at most 15 vertices, while Figure 1(b) shows the distribution of $c(p, r)$ under the same setting.[1] We omit the numerical results for $\lambda(p, r)$ since λ and c are closely related with $\lambda \leq c^4$ [30].Then, our observations are as follows:

– Figure 1(a) reveals a *unimodal* distribution of GEDs: the frequency rises rapidly, reaches a peak, and then decays gradually into a long tail. This pattern is consistent across different databases, suggesting an intrinsic property of the GED space.[2]

– Consider viewing Figure 1(a) from a single query graph's perspective: fixing a graph p, the curve reflects the distribution of GEDs from p to all other graphs [23]. From this perspective, the (grey) area under the curve up to radius r corresponds to $|B(p, r)|$, and then the ratio of the areas at $2r$ and r corresponds to $c(p, r) = |B(p, 2r)|/|B(p, r)|$. Such a distribution curve reveals a two-stage phenomenon of the GED space: *when r is small, $|B(p, 2r)|$ is much larger than $|B(p, r)|$, making $c(p, r)$ large; when r is large, $c(p, r)$ remains small and stable.* Figure 1(b) further illustrates this phenomenon by showing $c(p, r)$ against r, where three cases can be identified: Cases I and II are in a small-GED stage, where large or unstable

[1]We adopt this setting for the numerical evaluation since computing all pairwise exact GEDs for the entire dataset is too time-consuming.

[2]The distribution is consistent with that of approximate GED computed by GREED [37] on the entire AIDS dataset.

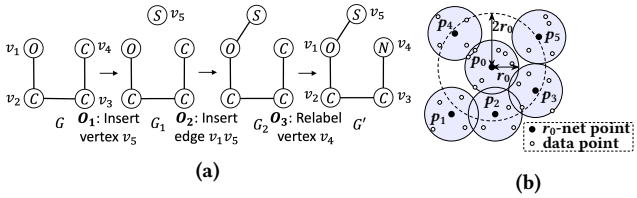


Figure 2: (a) A graph edit path $[O_1, O_2, O_3]$ that transforms graph G into graph G', resulting in a GED of 3; (b) An illustration of an r_0-net, where solid dots p_i, $0 \leq i \leq 5$ are r_0-net centers, and small circles are data points in database $\mathcal{D}$.

$c(p, r)$ makes doubling methods inefficient; Case III is in a large-GED stage, where $c(p, r)$ is small and stable and doubling methods become efficient. As the expansion constant $c = \max_{p,r} c(p, r)$ is dominated by the large values at small distances, it is inefficient to directly apply existing doubling metrics-based indexes.

• Gisma **index.** Building on the above insights, we propose a two-stage index called Gisma to achieve efficient query processing for approximate graph similarity search. The first stage, `NetDAG`, uses doubling metrics to locate the graphs when GED is larger than a radius threshold α, where the *restricted doubling constant* is denoted as $\lambda_\alpha = \max_{p, r \geq \alpha} \lambda(p, r)$. Specifically, `NetDAG` organizes the graph database into a hierarchical structure, where each ball B can be covered by B's child-layer balls having a radius equal to half of B's radius, which is in line with the doubling constant λ. This allows traversal by giant steps, efficiently pruning the entire search space to small subspaces (*i.e.,* balls in `NetDAG`). The second stage, namely `EditPathForest` (`EPF`), is built for GEDs smaller than α. `EPF` is a set of `EditPathTrees` (`EPTs`), each of which is a tree-like index constructed from the graph edit paths whose lengths equal GEDs between data graphs, and is used for *small-step* search within balls of radius α. Gisma is consistent with an observation on proximity graph-based indexes for ANNS search on non-graph data [15, 21, 25, 32, 42]: the search quickly approaches the query from a distant starting point, but converges slowly near it.

Contributions. The contributions of this paper are as follows:

- We analyze the GED space and find that its doubling/expansion constant is small at large distances but large at small distances, exhibiting a two-stage phenomenon. Accordingly, we propose a novel two-stage index called Gisma to achieve efficient approximate graph similarity search.
- For the first stage of Gisma, we propose `NetDAG`, a hierarchical index that supports *giant-step* traversals and achieves $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ worst-case query time, which is the lowest among those of existing doubling-based methods. Gisma uses `NetDAG` for the case of GED $\geq \alpha$ only to avoid the high computational complexity at small GEDs due to a large doubling constant.
- For Gisma's second stage, we propose a tree-like index called `EditPathForest` that indexes the graph edit paths used to compute the GED between graphs, which enables *small-step* searching within balls of radius α, together with three optimizations.
- We conduct comprehensive experiments on four widely-used benchmark datasets against six representative methods. The results show that Gisma outperforms these methods in almost all cases, and the speedup is more significant when τ is large.

Organization. The rest of this paper is organized as follows. Section 2 introduces the preliminaries and formally defines the problem. Section 3 analyzes the GED space. Sections 4 and 5 detail the first and the second stage of Gisma, respectively, while Section 6 presents a cost model for Gisma's construction. Section 7 presents comprehensive experimental evaluations and Section 8 reviews related work. Finally, Section 9 concludes the paper. Due to space limitations, some detailed proofs and additional experimental evaluations are presented in the appendix of the technical report [17].

2 PRELIMINARIES, PROBLEM STATEMENT AND SOLUTION OVERVIEW

In this section, we introduce the preliminaries of approximate similarity search in graph databases. Then, we present some background and an overview of our index Gisma. For clarity, we summarize the frequently used notations in Table 1.

2.1 Preliminaries

We study a database of undirected graphs with vertex labels but without edge labels, following prior work [8, 34–36]. This setting is compatible with existing graph similarity search methods that incorporate edge labels [7, 27, 41].

Graph. A graph is denoted by $G = (V, E, \ell)$, where V and E are the sets of vertices and undirected edges of G, respectively. The mapping $\ell : V \rightarrow L$ is a function that maps a vertex in V to a label. For clarity, we use $V(G)$ and $E(G)$ to denote the vertex set and edge set of a graph G, respectively.

Graph edit path and distance [8, 34–36, 43]. We consider five edit operations on graphs, including vertex insertion, vertex deletion, vertex relabeling, edge insertion, and edge deletion. Given two graphs G and G', a *graph edit path* (GEP) from G to G' is the *shortest* sequence of edit operations on G that transforms G into G'; and the *graph edit distance* (GED) between G and G', denoted by $d(G, G')$, is the length of the GEP from G to G', or equivalently the length of the GEP from G' to G by symmetry.

Example 2.1. Figure 2(a) shows a graph edit path from G to G' with the sequence $[O_1, O_2, O_3]$ of edit operations, where (i) O_1 is the vertex insertion of vertex v_5 with label S; (ii) O_2 is the edge insertion of edge (v_1, v_5); (iii) O_3 is vertex relabeling that changes v_4's label from C to N. As $[O_1, O_2, O_3]$ has the minimum length of 3, the graph edit distance between G and G' is $d(G, G') = 3$.

Computing the exact GED is NP-hard [44], which makes graph similarity search computationally costly on large databases. Therefore, in this paper, we focus on approximate graph similarity search, and propose an index-based solution to efficiently tackle this problem.

Problem statement. Given a graph database $\mathcal{D} = \{G_1, \ldots, G_N\}$, a query graph Q and a threshold τ, the *approximate graph similarity search* problem is to retrieve approximate answer graphs in $\mathcal{D}$, where the answer graphs G satisfy that $d(G, Q)$ (GED between G and Q) is at most τ. This paper aims to provide an efficient solution with a high recall.

Table 1: Frequently used notations

Notation	Description
$\mathcal{D}; Q; G$	A database of graphs; a query graph Q; a (data) graph G in $\mathcal{D}$
$d(G, G')$	The graph edit distance (GED) between two graphs G, G'
τ	The GED threshold for graph similarity search
(Q, τ)	A similarity search query with range τ
$(\mathcal{X}, d)$	A metric space where $\mathcal{X}$ is a set of points and d is the GED
$B(p, r)$	$\{p' \in \mathcal{X} \mid d(p, p') \leq r\}$, a set of points within distance r from p
r-ball	A ball $B(p, r)$ for some center p
$\lvert B(\cdot, \cdot)\rvert$	The number of points within a ball
$\lambda; c$	Doubling constant; expansion constant
$\lambda(p, r); c(p, r)$	Min. number of r-balls to cover $B(p,2r)$; $\lvert B(p,2r)\rvert / \lvert B(p,r)\rvert$
α	The radius threshold parameter of Gisma
$\lambda_\alpha; c_\alpha$	Restricted doubling/expansion constant given α
$V(G); E(G)$	The vertex set and edge set of a graph G
N	The number of graphs in the database
Y_i	The i-th layer of `NetDAG`, an $(\alpha \cdot 2^i)$-net of $\mathcal{D}$
ϕ	Number of layers in `NetDAG`
p_j^i	The point p_j in layer Y_i of `NetDAG`
$\mathcal{A}$	The answer set $\{g \in \mathcal{D} \mid d(g, Q) \leq \tau\}$
`NDC`	Number of distance computations

2.2 Background of Gisma

For ease of Gisma's illustration, we introduce several fundamental concepts in metric spaces, including the doubling constant [30], the expansion constant [4, 26], doubling spaces [30], and r-nets [22, 30]. Consider a set $\mathcal{X}$ of data points and a distance metric d.

Definition 2.1 (*Doubling constant* [30])**.** Given a metric space $(\mathcal{X}, d)$, the *doubling constant* λ is the smallest number such that every ball $B(p, 2r)$ can be covered by at most λ balls of radius r, for all $p \in \mathcal{X}$ and $r > 0$.

Remark. For a given p and r, we denote by $\lambda(p, r)$ the minimum number of r-balls (*i.e.,* balls of radius r) needed to cover $B(p, 2r)$. Then we have $\lambda = \max_{p \in \mathcal{X}, r > 0} \lambda(p, r)$. We denote by $\lambda_\alpha := \max_{p \in \mathcal{X}, r \geq \alpha} \lambda(p, r)$ the *restricted doubling constant* given radius threshold α.

Definition 2.2 (*Expansion constant* [4, 26])**.** Given a metric space $(\mathcal{X}, d)$, the *expansion constant* c is the smallest value $c \geq 2$ such that $|B(p, 2r)| \leq c \cdot |B(p, r)|$ for all $p \in \mathcal{X}$ and $r > 0$.

Remark. For a given p and r, we denote $c(p, r) = |B(p, 2r)|/|B(p, r)|$. Then $c = \max_{p \in \mathcal{X}, r > 0} c(p, r)$. We denote by $c_\alpha := \max_{p \in \mathcal{X}, r \geq \alpha} c(p, r)$ the *restricted expansion constant* given radius threshold α.

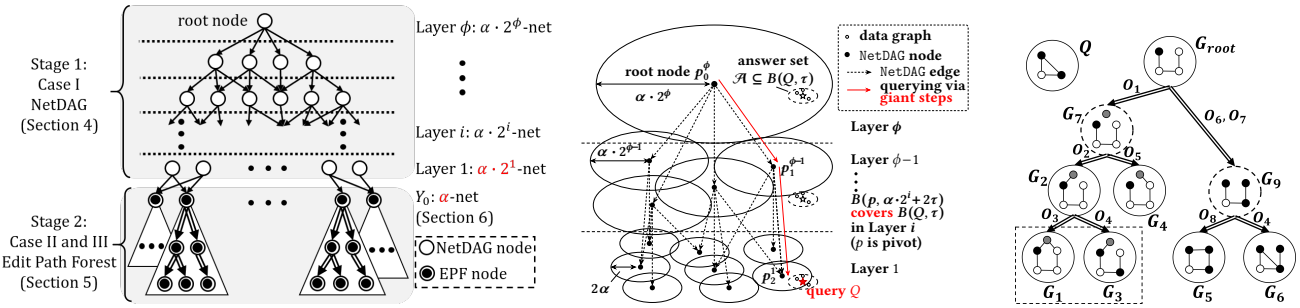
Definition 2.3 (*Doubling space* [22, 30])**.** A metric space $(\mathcal{X}, d)$ is *doubling* if its doubling constant λ is finite.

Example 2.2 (Computing $\lambda(p, r)$ and $c(p, r)$). In Figure 2(b), for p_0 and the radius r_0 shown in the figure, $B(p_0, 2r_0)$ can be covered by six r_0-balls, so $\lambda(p_0, r_0) = 6$. Moreover, $|B(p_0, r_0)| = 8$ and $|B(p_0, 2r_0)| = 19$. Hence, $c(p_0, r_0) = 19/8 = 2.38$.

Definition 2.4. (*r-net* [30]) Given a metric space $(\mathcal{X}, d)$ and a radius r, where $\mathcal{X}$ is a set of points and d is the distance measure, an *r-net* $Y \subseteq \mathcal{X}$ is a subset of $\mathcal{X}$ that satisfies the following:

- For $p_i, p_j \in Y$, $p_i \neq p_j \Rightarrow d(p_i, p_j) \geq r$; and
- $\mathcal{X} = \bigcup_{p \in Y} B(p, r)$, where $B(p, r) = \{p' \in \mathcal{X} \mid d(p, p') \leq r\}$.

Intuitively, the first condition ensures that the points in Y are *r-separated* [22] (*i.e.,* pairwise distance $\geq r$), and the second condition



(a) Overview of Gisma **(Stage 1:** `NetDAG`**; Stage 2:** `EditPathForest`**)** **(b) An example of** `NetDAG` **(c) An example of** `EditPathTree`

Figure 3: Overview of Gisma **– Stage 1:** `NetDAG` **(Section 4); and Stage 2:** `EditPathForest` **(Section 5). Both indices depend on the threshold α: the leaf layer of** `NetDAG` **is a 2α-net, and α is the radius of the balls indexed by** `EPT`**s.**

ensures that the collection of r-balls $\{B(p, r) \mid p \in Y\}$ covers the entire space $\mathcal{X}$.

Example 2.3. Figure 2(b) shows the space $\mathcal{X}$, where each node is a data point in $\mathcal{X}$. $Y = \{p_0, p_1, \ldots, p_5\}$ forms an r_0-net of $\mathcal{X}$. The balls of radius r_0 centered at p_i for $i \in \{0, \ldots, 5\}$ together cover all points, and the centers satisfy $d(p_i, p_j) \geq r_0$ for any $i \neq j$.

Definition 2.5. (*Greedy permutation* [22]) Given a metric space $(\mathcal{X}, d)$ with $N = |\mathcal{X}|$, a *greedy permutation* of $\mathcal{X}$ is an ordering $\langle p_1, p_2, \ldots, p_N \rangle$ of the points in $\mathcal{X}$ such that for every $2 \leq k \leq N$, $p_k = \arg\max_{p \in \mathcal{X}} \min_{1 \leq i < k} d(p, p_i)$.

For each k, let $r_k = \min_{1 \leq i < k} d(p_k, p_i)$ denote the distance from p_k to its nearest earlier center. Importantly, the prefix $\langle p_1, \ldots, p_k \rangle$, as a set $\{p_1, \ldots, p_k\}$, forms an r_k-net of $\mathcal{X}$ (Definition 2.4). The detailed construction procedure is given in Appendix 10.1.3.

2.3 Gisma Overview

An overview of Gisma is presented in Figure 3(a). Gisma's first stage index `NetDAG` consists of multiple layers of nodes satisfying that (i) each layer is an r-net (Definition 2.4) with a different radius r, (ii) for adjacent layers, the child layer's radius is half of the parent layer's radius, and (iii) there exists an edge from a parent node p to a child node p' if the GED between p and p' is within a certain distance (detailed in Section 4). The resulting structure is a *DAG*, and we denote `NetDAG`'s 2α-net Y_1 as the leaf layer.

Regarding the second stage index `EditPathForest` (`EPF`), it consists of a set of `EditPathTrees` (`EPT`s) such that (i) their roots are data graphs in $\mathcal{D}$, forming an α-net Y_0, where α is the radius threshold for separating Gisma's two stages (Section 6); and (ii) each `EPT` indexes the graph edit paths from its root p to the graphs in $B(p, \alpha)$. Each point in Y_1 has edges to points in Y_0 within a certain distance (same rule as between layers in `NetDAG`), bridging the two stages.

To answer graph similarity search queries, we first traverse `NetDAG` top-down, selecting at each layer a single child of the current pivot close to Q. Upon reaching the leaf layer, the α-balls of the pivot's children in Y_0 are guaranteed to jointly cover all query answers. We then collect the candidate balls among them, and perform a depth-first search on their corresponding `EPT`s, with three optimizations to improve the query efficiency.

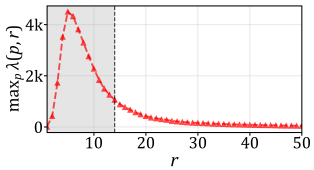


Figure 4: $\max_p \lambda(p, r)$ **is small when r is large (bright area) and sensitive to r when r is small (dark area).**

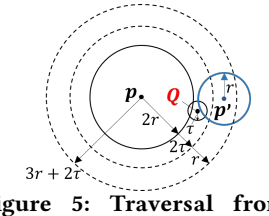


Figure 5: Traversal from pivot p ($d(p, Q) \leq 2r + \tau$) to child $p' \in B(p, 3r + 2\tau)$ with $d(p', Q) \leq r + \tau$, where $r = \alpha \cdot 2^{i-1}$.

3 ANALYSIS OF GED SPACE

In this section, we show that the GED space is a doubling space, and analyze the corresponding doubling constant.

Lemma 3.1. *Given the set $\mathcal{X}$ of graphs with at most n vertices and at most ρ distinct labels, where n and ρ are constants, the metric space $(\mathcal{X}, \text{GED})$ induced by the graph edit distance is a doubling space.*

With n and ρ fixed, the number of possible graphs is finite; let M denote this number. For any ball $B(p, 2r)$ in $(\mathcal{X}, \text{GED})$, it can be covered by at most $|B(p, 2r)|$ balls of radius r, each centered at a graph in $B(p, 2r)$. Since the number of graphs in $B(p, 2r)$ never exceeds M, the doubling constant is at most M. Therefore, $(\mathcal{X}, \text{GED})$ is a doubling space.

Since existing doubling-based indexes [22, 30] have query complexities of $O(\lambda^{O(1)} \log N)$ (where λ is the doubling constant and N is the dataset size), their efficiency highly depends on the value of λ, as λ is closely related to the fanout of each node in the index structure. Lemma 3.1 shows that the GED space is doubling, which enables doubling-based indexes. However, the query efficiency depends on the practical value of λ. We then analyze $\lambda(p, r)$ below.

Net-tree [22] proposed a method to estimate the global doubling constant λ, but it does not reveal how $\lambda(p, r)$ varies with r. To capture this, on top of the method in [22], we additionally record, for each integer radius r, the maximum number of r-balls required to cover any $2r$-ball, denoted as $\max_p \lambda(p, r)$. Due to the high computational cost of exact GED, we use GREED [37] to approximate GED in this process. The doubling constant is the peak value of the $\max_p \lambda(p, r)$ curve, which is equal to $\max_{p,r} \lambda(p, r)$ (Figure 4, on the AIDS dataset). In addition, we observe a sharp peak and a long tail in Figure 4.

4 NetDAG: HIERARCHICAL DOUBLING-BASED INDEX FOR GRAPHS HAVING LARGE GEDS

In this section, we propose NetDAG, a hierarchical doubling-based index for graphs having large GEDs, defined in Definition 4.1. Then, we describe how NetDAG is constructed from the database and present its query algorithm. Finally, we compare NetDAG with other existing doubling-based indexes to demonstrate that it achieves the lowest query time complexity.

Definition 4.1 (NetDAG)**.** Given a graph database of N points $\mathcal{D} = \{p_0, p_1, \ldots, p_{N-1}\}$, a threshold $\alpha > 0$, a constant τ with $0 \le \tau \le \alpha$, and an integer $\phi \ge 0$, a NetDAG is a ϕ-layered DAG $H = (Y_1, Y_2, \ldots, Y_\phi, E)$, where:

- Each layer Y_i is an $(\alpha \cdot 2^i)$-net (Definition 2.4) of $\mathcal{D}$;
- $E = \{(p_j^{i+1}, p_k^i) \mid d(p_j^{i+1}, p_k^i) \le 3\alpha \cdot 2^i + 2\tau\}$, where $i = 1, \ldots, \phi - 1$ and p_k^i denotes the point p_k in layer Y_i.

Remarks. The top layer is Y_ϕ, and the leaf layer is Y_1. For any node $p_j^{i+1} \in Y_{i+1}$, its children are children$(p_j^{i+1}) := \{\, p_k^i \in Y_i \mid (p_j^{i+1}, p_k^i) \in E \,\}$.

Since a node in Y_i may be connected to multiple nodes in Y_{i+1}, H is a DAG rather than a tree. As in doubling-based indexes [4, 22, 30], queries are answered by traversing H top-down. Letting $r = \alpha \cdot 2^i$ be the radius of the child layer, the edge range $3r + 2\tau$ is *wider* than that of some existing methods (*e.g.,* Cover Tree [4] uses $2r$). While this increases the number of children per node, it allows the query algorithm to maintain only one pivot at each layer rather than a set of candidates as in existing methods (Corollary 1), yielding a lower per-layer query complexity (Theorem 4.6).

4.1 Index Construction

We construct NetDAG based on the greedy permutation (Definition 2.5) of the dataset. Since every prefix $\langle p_1, \ldots, p_k \rangle$, considered as a set $\{p_1, \ldots, p_k\}$, forms an r_k-net, each layer Y_i of NetDAG is obtained by taking the prefix at the point when r_k first satisfies $r_k \le \alpha \cdot 2^i$. When the radius decreases from $r_{k-1} > \alpha \cdot 2^i$ to $r_k \le \alpha \cdot 2^i$, we generate the $(\alpha \cdot 2^i)$-net layer $Y_i := \{p_1, \ldots, p_{k-1}\}$. When a layer Y_i ($i \ge 1$) is generated, we assign to each center $q \in Y_{i+1}$ its children drawn from Y_i: an edge is added for every $p_\ell \in Y_i$ satisfying $d(p_\ell, q) \le 3\alpha \cdot 2^i + 2\tau$. The lowest layer is Y_1.

Beyond Y_1, the greedy permutation continues to produce the α-net Y_0, whose points serve as the roots of EPTs. For each point in Y_1, its children in Y_0 are determined using the same edge rule $d(q, p) \le 3\alpha + 2\tau$ (see Appendix 10.1.3 for details).

Analysis of NetDAG space and construction time complexity. Following Net-Tree [22], NetDAG is built on the greedy permutation. Although NetDAG proposes a different edge rule between layers, this does not increase the construction cost. Net-Tree's construction requires $O(\lambda^{O(1)} N s)$ distance computations [22], where λ is the global doubling constant and s is the number of layers. Since NetDAG terminates at radius α (with $\phi \le s$ layers), and all radii involved satisfy $r \ge \alpha$, the complexity reduces to $O(\lambda_\alpha^{O(1)} N \phi)$ for construction and $O(\lambda_\alpha^{O(1)} N)$ for space, where λ_α is the doubling constant restricted to radii $\ge \alpha$. The proofs are given in Appendix 10.1.

Algorithm 1: Giant-Step Query Processing on NetDAG

Input: H: NetDAG $(Y_1, \ldots, Y_\phi, E)$; Y_0: α-net; Q: query graph; τ: threshold
Output: $\mathcal{P}$: set of entry points in Y_0, or $\emptyset$ if no answer exists

```
1  P ← ∅;
2  p ← root of NetDAG;                          // initialize pivot
3  for i = φ − 1 to 1 do                        // from root layer to leaf layer
4      found ← false;
5      for p_j^i such that (p, p_j^i) ∈ E do
6          if d(p_j^i, Q) ≤ α · 2^i + τ then
7              p ← p_j^i;                       // giant step: roughly halve d(p, Q)
8              found ← true;
9              break;
10     if not found then return ∅;
11 for p_j^0 such that d(p, p_j^0) ≤ 3α + 2τ do   // collect candidate balls
12     if d(p_j^0, Q) ≤ α + τ then
13         P ← P ∪ {p_j^0};
14 return P;
```

4.2 Querying via Giant Steps

The giant-step query traverses the layers from Y_ϕ to Y_1, selecting a pivot at each layer. Let $\mathcal{A} = \{g \in \mathcal{D} \mid d(g, Q) \le \tau\}$ be the answer set. At each layer Y_i, $\mathcal{A}$ is guaranteed to be contained in a ball centered at the pivot p with radius $\alpha \cdot 2^i + 2\tau$. The distance from the pivot to Q decreases from at most $\alpha \cdot 2^\phi + \tau$ at the top to at most $2\alpha + \tau$ at the leaf, roughly halving at each layer. Algorithm 1 presents the query algorithm for NetDAG. Starting with the root of NetDAG as the initial pivot p in the top layer Y_ϕ (Line 2), it iteratively traverses the layers. At each layer i, it searches for children of the current pivot p that satisfy the distance constraint $d(p_j^i, Q) \le \alpha \cdot 2^i + \tau$ (Lines 5–6). Algorithm 1 takes the first child that satisfies the distance constraint as the new pivot and proceeds to the next layer (Lines 7–9), without the need to check the remaining children. Each such step roughly halves $d(p, Q)$, rapidly narrowing the search space (Figure 5). If no such child exists, the algorithm returns empty (Line 10). Upon reaching the leaf layer Y_1, the algorithm collects the points p_j^0 in Y_0 satisfying $d(p, p_j^0) \le 3\alpha + 2\tau$ and $d(p_j^0, Q) \le \alpha + \tau$ as candidate ball centers $\mathcal{P}$ for the subsequent EditPathForest search (Lines 11–13).

Lemma 4.1. *Given a* NetDAG*, for any node* $p^{i+1} \in Y_{i+1}$, $B(p^{i+1}, \alpha \cdot 2^{i+1} + 2\tau) \subseteq \bigcup_{p^i \in \text{children}(p^{i+1})} B(p^i, \alpha \cdot 2^i)$.

Proof. Let x be an arbitrary point in $B(p^{i+1}, \alpha \cdot 2^{i+1} + 2\tau)$. Since Y_i is an $(\alpha \cdot 2^i)$-net of $\mathcal{D}$, there exists $p^i \in Y_i$ such that $d(x, p^i) \le \alpha \cdot 2^i$, and thus $x \in B(p^i, \alpha \cdot 2^i)$. Furthermore, $d(p^{i+1}, p^i) \le d(p^{i+1}, x) + d(x, p^i) \le (\alpha \cdot 2^{i+1} + 2\tau) + \alpha \cdot 2^i = 3\alpha \cdot 2^i + 2\tau$. By Definition 4.1, this implies $(p^{i+1}, p^i) \in E$, so $p^i \in$ children(p^{i+1}). □

Corollary 1. Given a NetDAG, if $\mathcal{A} \ne \emptyset$ and $\mathcal{A} \subseteq B(p^{i+1}, \alpha \cdot 2^{i+1} + 2\tau)$ for some $p^{i+1} \in Y_{i+1}$, then there exists a child $p^i \in$ children(p^{i+1}) such that $d(p^i, Q) \le \alpha \cdot 2^i + \tau$.

Theorem 4.2. *In Algorithm 1, at each layer i* ($i = \phi, \ldots, 1$)*, the pivot p satisfies* $\mathcal{A} \subseteq B(p, \alpha \cdot 2^i + 2\tau)$.

Proof. We prove by induction on i from ϕ to 1.

Base case ($i = \phi$). Since the top layer $Y_\phi = \{p\}$ is an $(\alpha \cdot 2^\phi)$-net of $\mathcal{D}$, every query answer is within distance $\alpha \cdot 2^\phi$ from p, so $\mathcal{A} \subseteq B(p, \alpha \cdot 2^\phi + 2\tau)$.

Inductive step. Assume $\mathcal{A} \subseteq B(p,\ \alpha \cdot 2^{i+1} + 2\tau)$ at layer $i+1$ for some $i+1 \geq 2$. If $\mathcal{A}$ is empty, the claim holds trivially. Otherwise, Corollary 1 ensures that there exists a child p' of p satisfying $d(p', Q) \leq \alpha \cdot 2^i + \tau$. By the triangle inequality, for any $g \in \mathcal{A}$, $d(g, p') \leq d(g, Q) + d(Q, p') \leq \tau + (\alpha \cdot 2^i + \tau) = \alpha \cdot 2^i + 2\tau$, so $\mathcal{A} \subseteq B(p',\ \alpha \cdot 2^i + 2\tau)$. □

Theorem 4.3. *At layer i of Algorithm 1, if a child p' of the current pivot p satisfies $d(p', Q) > \alpha \cdot 2^i + \tau$, then the child ball $B(p', \alpha \cdot 2^i)$ contains no query answer $g \in \mathcal{A}$.*

Proof. For any $g \in \mathcal{A}$, by the triangle inequality, $d(p', g) \geq d(p', Q) - d(g, Q) > \alpha \cdot 2^i$, so $g \notin B(p', \alpha \cdot 2^i)$. □

Remark. Theorem 4.3 guarantees the correctness of returning $\emptyset$ in Line 10 of Algorithm 1. By Theorem 4.2, $\mathcal{A} \subseteq B(p,\ \alpha \cdot 2^{i+1} + 2\tau)$, which by Lemma 4.1 is covered by its child balls. If no child p' satisfies $d(p', Q) \leq \alpha \cdot 2^i + \tau$, then by Theorem 4.3, none of these child balls contains any $g \in \mathcal{A}$, so $\mathcal{A} = \emptyset$.

Theorem 4.4. *Let p be the pivot returned by Algorithm 1 at layer 1, and let $\mathcal{P} = \{\, p_j^0 \in Y_0 \mid d(p, p_j^0) \leq 3\alpha + 2\tau$ and $d(p_j^0, Q) \leq \alpha + \tau \,\}$. Then $\mathcal{A} \subseteq \bigcup_{p_j^0 \in \mathcal{P}} B(p_j^0,\ \alpha)$.*

Proof. For any $g \in \mathcal{A}$, by Theorem 4.2 with $p \in Y_1$, $d(g, p) \leq 2\alpha + 2\tau$. Since Y_0 is an α-net, there exists $p_j^0 \in Y_0$ with $d(g, p_j^0) \leq \alpha$, so $g \in B(p_j^0, \alpha)$. By the triangle inequality, $d(p, p_j^0) \leq d(p, g) + d(g, p_j^0) \leq (2\alpha + 2\tau) + \alpha = 3\alpha + 2\tau$, and $d(p_j^0, Q) \leq d(p_j^0, g) + d(g, Q) \leq \alpha + \tau$, so $p_j^0 \in \mathcal{P}$ and $g \in \bigcup_{p_j^0 \in \mathcal{P}} B(p_j^0, \alpha)$. □

Example 4.5. To illustrate the giant-step query (Figure 3(b)), we set $\phi = 3$, giving three layers $Y_\phi, Y_{\phi-1}$, and Y_1. Let the root pivot be $p = p_0^\phi \in Y_\phi$, which satisfies $d(p, Q) \leq \alpha \cdot 2^\phi + \tau$. Then $B(p,\ \alpha \cdot 2^\phi + 2\tau)$ covers $B(Q, \tau)$ (contains $\mathcal{A}$). By Corollary 1, the algorithm selects a child $p_1^{\phi-1} \in Y_{\phi-1}$ as the new pivot p, satisfying $d(p, Q) \leq \alpha \cdot 2^{\phi-1} + \tau$. The ball $B(p,\ \alpha \cdot 2^{\phi-1} + 2\tau)$ still covers $B(Q, \tau)$. Similarly, the algorithm selects $p_2^1 \in Y_1$ as the new pivot p, satisfying $d(p, Q) \leq 2\alpha + \tau$. The ball $B(p,\ 2\alpha + 2\tau)$ covers $B(Q, \tau)$. The children p_j^0 of p in Y_0 satisfying $d(p_j^0, Q) \leq \alpha + \tau$ are collected as candidate ball centers for the `EditPathForest` search (Theorem 4.4).

Theorem 4.6 (Time Complexity of Querying `NetDAG`). *Given a `NetDAG` with ϕ layers, Algorithm 1 completes in $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ distance computations (NDCs).*

Proof idea. At each layer the pivot has at most $\lambda_\alpha^{\log_2 20}$ children (detailed in Corollary 10.2 of Appendix 10), and the algorithm checks at most this many of them to find the next pivot. Since there are ϕ layers, the overall complexity is $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$. A detailed proof is deferred to Appendix 10.

4.3 Analysis of `NetDAG` and Other Doubling-Based Indexes

We compare `NetDAG` with other representative doubling-based indexes for non-graph data [4, 22, 30] and summarize the results in Table 2. Since `NetDAG` serves as the first stage of Gisma and narrows down the query answers to several α-balls in `EditPathForest`, we compare these methods under the same setting: all methods build a hierarchical structure above the same α-net Y_0 and traverse top-down to find the candidate α-balls in Y_0. For fair comparison, the ratio of the radius of the parent layer to its child layer is $\kappa = 2$, for all methods. Thus, the radius of layer i is $r_i = \alpha \cdot 2^i$. All methods have the same number of layers ϕ, and r_i is written as r in Table 2 since the table holds for all $1 \leq i \leq \phi - 1$. For methods that give only asymptotic bounds of the form $\lambda^{O(1)}$, we derive specific constants using Lemma 10.1 and Corollary 2 in Appendix 10.1.2.

The efficiency of the traversal is thus determined by the per-layer NDC. At each layer, the traversal maintains a candidate set and expands the children of each candidate, so the per-layer NDC is the product of the *candidate set size* and the *number of children per node*. In a doubling space, these two values are bounded by their range (ball radius containing the set) and separation (minimum pairwise distance) respectively, as shown in Table 2. In `NetDAG`, the child range $3r + 2\tau$ includes a 2τ margin that guarantees the coverage of the answer ball $B(Q, \tau)$, so that at least one child satisfies $d(p, Q) \leq r + \tau$ (Corollary 1). This allows maintaining only a single pivot rather than a candidate set (Algorithm 1), reducing the per-layer NDC to $\lambda_\alpha^{\log_2 20}$.

5 EDIT PATH FOREST: PATH-BASED INDEX FOR GRAPHS HAVING SMALL GEDS

Recall from Figures 1(a) and 4 that when the GED between graphs is small, doubling-based indexes can be inefficient or sensitive to r. To address this, we exploit the fact that graphs with small GEDs differ by only a few edit operations, and their edit paths from a common root share prefixes. Based on this, we propose `EditPathForest` (`EPF`), a prefix-tree-based index that shares common edit paths among nearby graphs, enabling path-based pruning and search in *small steps*. The `EPF` is a set of `EditPathTrees` (`EPT`s). Each `EPT` has a root that is a center p in the α-net Y_0. `EPT` indexes the graphs that (i) are covered by the ball $B(p, \alpha)$ *and* (ii) have p as their nearest center in Y_0. Specifically, `EditPathTree` (`EPT`) is defined below.

Definition 5.1. Given a set of graphs $C = \{G_1, G_2, \ldots, G_m\}$ that are covered by $B(G_{\text{root}}, \alpha)$, a `EditPathTree`, denoted as $\mathcal{T}$, is a prefix tree rooted at G_{root} (which is a graph in C) that indexes the graph edit paths (GEPs) from G_{root} to the graphs in C, as follows.

- Each node G_i in $\mathcal{T}$ is a graph, and each edge represents the application of edit operation(s) from the parent graph to the child graph. A path from G_{root} to G_i is the sequence of edit operations that transforms G_{root} into G_i.
- Each node G_i in $\mathcal{T}$ that belongs to C is called a *data node*; otherwise, it is a *non-database node*.

The construction algorithm of `EPT` is presented in Algorithm 2. We compute the graph edit paths (GEPs) from the root graph p to the graphs $C = \{G_1, G_2, \ldots, G_m\}$ in $B(p, \alpha)$ (Line 2), organize the GEPs into a prefix tree (Line 3), and compress the tree by removing single-child non-database nodes (Line 4). For each ball center in the α-net Y_0, we apply Algorithm 2 to obtain its `EPT`.

Analysis of `EPF` space and construction time complexity. Let P be the set of roots of the `EPT`s in `EPF`, and let $C_p \subseteq \mathcal{D}$ be the set of data graphs indexed by the `EPT` rooted at $p \in P$. The worst-case

Table 2: Comparison of NetDAG and representative doubling-based methods ($\kappa = 2$)

Index	Child Range	Child Separation	# Children	Index traversal proceeds to	Cand. Range	Cand. Separation	Size of cand. set	Dist. to Q	NDC / layer
NetDAG	$3r + 2\tau$	r	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)	ONE child	N/A	N/A	1	$\leq r + \tau$	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)
Net-Tree [22]	$4r$	r	λ_α^4	all children	$13r$	r	$\lambda_\alpha^{\log_2 52}$	$\leq 4r + \tau$	$\lambda_\alpha^{4+\log_2 52}$
Navigating Nets [30]	$8r$	r	λ_α^5	all children	$d_{\min}(Q, \text{cand_set}) + 2r$	r	$\lambda_\alpha^{\log_2 28}$	$\leq 2r + \tau$	$\lambda_\alpha^{5+\log_2 28}$
Cover Tree [4]	$2r$	r	c_α^4	all children	$d_{\min}(Q, \text{cand_set}) + 2r$	r	c_α^5	$\leq 2r + \tau$	c_α^9

Algorithm 2: Construction of EditPathTree

Input: G_{root}: root (center) graph; C: data graphs to be indexed
Output: $\mathcal{T}$: constructed EditPathTree

1 Initialize $\mathcal{T}$ with root G_{root};
2 $P \leftarrow$ ComputeGEPs(G_{root}, C);
3 Build a prefix tree $\mathcal{T}$ from all GEPs in P;
4 Compress $\mathcal{T}$ by removing single-child non-database nodes;
5 **return** $\mathcal{T}$;
6 **Function** ComputeGEPs(G_{root}, C)
7 | **for** $G_i \in C$ **do**
8 | | $P[G_i] \leftarrow$ the GEP from G_{root} to G_i;
9 | **return** P;

space of the EPT rooted at p is $O(|C_p| \cdot |E(p)| + \sum_{G \in C_p} |E(G)|)$ (Corollary 3 in Appendix). Summing over all EPTs, the total space of EPF is $O(\sum_{p \in P} |C_p| \cdot |E(p)| + \sum_{G \in \mathcal{D}} |E(G)|)$. For construction, Algorithm 2 computes one GEP per data graph (Line 2), so the total number of GEP computations is $\sum_{p \in P} |C_p| = N$.

Example 5.1. Figure 3 (c) is an example of EPT. G_{root} is the center of the ball. $\{G_1, G_2, G_3, G_4, G_5, G_6\}$ are the data graphs. G_2 is a data graph in C but is not a leaf node. G_9 is a *compressed node* that represents two edit operations. The EPT does not contain all data graphs in the ball, since some may be closer to another center and are assigned to other EPTs.

Remarks. There is a subtle technical detail at the boundary of NetDAG and EPF. We briefly describe how NetDAG is connected to EPF. As shown in Section 4.1, during the construction of NetDAG, the greedy permutation generates the leaf layer of NetDAG (*i.e.,* Y_1), as well as an α-net Y_0. As in Definition 4.1, $p^1 \in Y_1$ has an edge to $p^0 \in Y_0$ that satisfies $d(p^0, p^1) \leq 3\alpha + 2\tau$. For each p^0, we build a EPT. When the query algorithm (Section 4.2) reaches p^1, the EPTs of its children are searched further.

5.1 Querying via Small Steps

Given the selected points $p_i \in Y_0$ and their balls $B(p_i, \alpha)$ identified by the NetDAG, the search is restricted to the EPTs rooted at p_i. For each EPT, a depth-first search (DFS) is performed to retrieve all database graphs whose distance to the query graph Q is within τ. To improve query efficiency, we introduce three optimizations, namely, (1) EPT *Subtree Pruning*; (2) *Lower-bound Pruning and its Propagation*; and (3) *Search Tree Reuse in* GED *Computations.*

5.1.1 EPT *Subtree Pruning.* To reduce the number of GED computations, we propose EPT *subtree pruning* to prune the subtrees of an EPT that cannot contain answers. Let $\bar{d}_G$ denote the maximum GED from G to any of its descendants. The minimum possible distance between any node in the subtree rooted at G and the query Q is $d(G, Q) - \bar{d}_G$. By the triangle inequality of GED,

Algorithm 3: Small-step Query Processing on EPT

Input: $\mathcal{T}$: EditPathTree; Q: query graph; τ: threshold
Output: $\mathcal{A}$: set of data graphs within distance τ

1 $\mathcal{A} \leftarrow \emptyset$;
2 initialize a stack S with ($\mathcal{T}$.root, 0)
3 **while** $S \neq \emptyset$ **do**
4 | $(G, \underline{d}) \leftarrow$ pop(S); // small step: traverse along edit paths
5 | $\bar{d}_G \leftarrow \max\{\, d(G, G_d) \mid G_d$ is a descendant of G in $\mathcal{T}\}$;
6 | **if** $\underline{d} - \bar{d}_G > \tau$ **then**
7 | | **continue**; // 5.1.1 subtree pruning
8 | **if** $\underline{d} > \tau$ **then**
9 | | **for** G_c is a child of G **do**
10 | | | $\underline{d}_c \leftarrow \underline{d} - d(G, G_c)$; // 5.1.2 lower-bound propagation
11 | | | push($(G_c, \underline{d}_c), S$);
12 | | **continue**;
13 | $d \leftarrow d(G, Q)$; // compute exact GED
14 | **if** $d - \bar{d}_G > \tau$ **then**
15 | | **continue**; // 5.1.1 subtree pruning
16 | **if** G is a data graph and $d \leq \tau$ **then**
17 | | add G to $\mathcal{A}$;
18 | **for** G_c is a child of G **do**
19 | | $\underline{d}_c \leftarrow d - d(G, G_c)$; // 5.1.2 lower-bound propagation
20 | | push($(G_c, \underline{d}_c), S$);
21 **return** $\mathcal{A}$;

$d(G, Q) \leq d(G, G') + d(G', Q)$. If $d(G, Q) - \bar{d}_G > \tau$, the subtree rooted at G can be pruned, because for any graph G' that is a descendant of G, the distance from G' to Q is greater than τ.

5.1.2 Lower-bound Pruning and its Propagation. For an edge (G_p, G_c) of EPT, once $d(G_p, Q)$ is computed, we define $\underline{d}(G_c, Q) = d(G_p, Q) - d(G_p, G_c)$. Also, $\underline{d}(G_c, Q) \leq d(G_c, Q)$. If $\underline{d}(G_c, Q) > \tau$, then G_c cannot be an answer and its exact GED computation is skipped. When the exact $d(G_p, Q)$ is not computed, the lower bound can still be *propagated*: we set $\underline{d}(G_c, Q) = \underline{d}(G_p, Q) - d(G_p, G_c)$, without computing the exact GED $d(G_p, Q)$. Similarly, if $\underline{d}(G_c, Q) > \tau$, G_c cannot be an answer. This propagation can be applied *recursively* to skip exact GED computations along a chain of descendants.

Algorithm 3 presents the query procedure on the EPT. It performs a DFS traversal while applying subtree pruning based on both the lower bound and the exact GED (Lines 6–7 and Lines 14–15), lower-bound pruning (Line 8), and lower-bound propagation (Lines 10 and 19). Specifically, during the traversal, if the lower bound on the distance at a node exceeds the threshold, the algorithm skips the exact GED computation at this node (Line 8) and propagates the lower bound to its children for further pruning (Line 10). In addition, the exact GED computation is performed only when the propagated lower bound does not exceed the query threshold (Line 13), thereby reducing the total number of GED computations.

Example 5.2. Consider the EPT shown in Figure 3(c). In this example, the query threshold is set to $\tau = 2$. G_{root} has $d(G_{\text{root}}, Q) = 4$ and $\bar{d}_{G_{\text{root}}} = 3$, giving $d(G_{\text{root}}, Q) - \bar{d}_{G_{\text{root}}} = 1 \leq \tau$, so its subtree remains

to be explored. For the child G_7, the lower bound propagated from G_{root} is $\underline{d}(G_7, Q) = 3$. Since $\underline{d}(G_7, Q) - \bar{d}_{G_7} = 1 < \tau$, the subtree of G_7 is not pruned and continues to be searched. Meanwhile, as $\underline{d}(G_7, Q) > \tau$, the exact distance computation at G_7 is skipped by lower-bound pruning (Line 8). For its child G_2, the lower bound is propagated as $\underline{d}(G_2, Q) = \underline{d}(G_7, Q) - d(G_7, G_2) = 2 \leq \tau$, which leads to an exact computation of $d(G_2, Q) = 6$. Since $d(G_2, Q) - \bar{d}_{G_2} = 5 > \tau$, the subtree rooted at G_2 is pruned (Line 14).

5.1.3 Search Tree Reuse in GED *Computations.* We first recall that the A* search algorithm [6, 7, 28, 41] has been widely adopted for computing GED, which formulates GED computation as a node-matching problem. In these methods, A* organizes (partial) node mappings into a *search tree*, and expands this tree to obtain a full mapping with the minimum edit cost. Each *state* on the search tree corresponds to *a (partial) mapping between the two graphs.*

It should be noted that for two similar graphs G and G' (*i.e.,* having small $d(G, G')$) with the same number of nodes, their search trees against the same query graph Q often have large overlaps. In an EPT, a parent–child pair of indexed graphs typically consists of two similar graphs. We focus on the case where such parent–child graphs also have the same number of nodes, and propose a *search tree reuse* optimization that reuses the search tree constructed during the computation of $d(G, Q)$ to accelerate that of $d(G', Q)$.

To enable reuse, we retain from the search tree of $d(G, Q)$ only the states that are potentially useful for computing $d(G', Q)$. We denote by $d(G, Q|s)$ the edit distance from G to Q that completes the partial mapping at state s, and by $\underline{c}(G, Q|s)$ its lower bound. It should be remarked that $\underline{c}(G, Q|s) \leq \underline{c}(G, Q|s')$ if s' is a leaf in the subtree rooted at s in the search tree. After finishing $d(G, Q)$, we construct a *search tree* S_G for reuse by keeping every leaf state s of the search tree such that $\underline{c}(G, Q|s) \leq \tau + d(G, G')$ (*i.e.,* $\underline{c}(G, Q|s) - d(G, G') \leq \tau$), together with all its ancestor states. Due to the triangle inequality, given s we have $d(G', Q|s) \geq d(G, Q|s) - d(G, G') \geq \underline{c}(G, Q|s) - d(G, G')$. Hence, if $\underline{c}(G, Q|s) - d(G, G') > \tau$, then $d(G', Q|s) > \tau$ and the search subtree rooted at s can be pruned. In contrast, resuming the search from only the leaf nodes of S_G does not miss any answers. For presentation simplicity, when the graphs are clear from the context, we simply write $d(s)$ and $\underline{c}(s)$.

Algorithm 4 presents the A*-based GED computation with search tree reuse. Line 1 initializes the minimum edit cost $c_{\min}$ as τ. H (Line 2) is a priority queue that maintains the states that may lead to a full mapping at a cost at most $c_{\min}$. The algorithm initializes the *frontier states* of S_G (Line 3). The search *resumes* from these frontier states by computing their lower bounds $\underline{c}$, *e.g.,* by using [7, 41] (Line 5). In Lines 6–7, if $\underline{c}(s) \leq c_{\min}$, the frontier state s is pushed into the priority queue H. In practice, this helps to reduce the value of $c_{\min}$ faster, and consequently reduces the number of states pushed into H. In each iteration (Lines 8–16), the algorithm pops a state s from H (Line 9) that can lead to smaller cost than $c_{\min}$. If s corresponds to a full mapping, $c_{\min}$ is updated (Lines 10–11); otherwise, the algorithm expands s and pushes children s's, where $\underline{c}(s') \leq c_{\min}$ (Lines 13–16). The search terminates once H is empty (Line 8), and it returns the $c_{\min}$ found (Line 17).

Example 5.3. Figure 6 shows an example of search tree reuse. We have computed $d(G_9, Q)$ and obtained the search tree S_{G_9}, as illustrated in Figure 6(a), which is the input to Algorithm 4. G' is

Algorithm 4: GED Computation with Search Tree Reuse

Input: S_G: a reused search tree from (Q, G); Q: query graph; G': data graph; τ: threshold
Output: $d(G', Q)$: the GED between G' and Q

1 $c_{\min} \leftarrow \tau$ $\quad \Delta_{c_{\min}} \leftarrow$ false ; `// initialize min. edit cost`
2 $H \leftarrow$ an empty priority queue, in ascending order of $\underline{c}(s)$;
3 $L \leftarrow$ frontiers of S_G;
4 **foreach** $s \in L$ **do**
5 $\quad \underline{c}(s) \leftarrow$ `App-BMao-resume`(s, G', Q); `// update lower bound cost at s, e.g., [7, 41]`
6 $\quad$ **if** $\underline{c}(s) \leq c_{\min}$ **then**
7 $\quad\quad$ push(s, H);
8 **while** $H \neq \emptyset$ and $\underline{c}(\text{top}(H)) \leq c_{\min}$ **do**
9 $\quad s \leftarrow$ pop(H);
10 $\quad$ **if** s corresponds to a full mapping (from G' to Q) **then**
11 $\quad\quad c_{\min} \leftarrow \underline{c}(s)$ $\quad \Delta_{c_{\min}} \leftarrow$ true; `// update min. edit cost`
12 $\quad\quad$ **continue**;
13 $\quad$ **foreach** $s' \in$ child(s) **do**
14 $\quad\quad \underline{c}(s') \leftarrow$ `App-BMao-resume`(s', G', Q); `// update lower bound cost at s'`
15 $\quad\quad$ **if** $\underline{c}(s') \leq c_{\min}$ **then**
16 $\quad\quad\quad$ push(s', H);
17 **if** $\Delta_{c_{\min}}$ **then return** $c_{\min}$ **else return** `"undefined"`

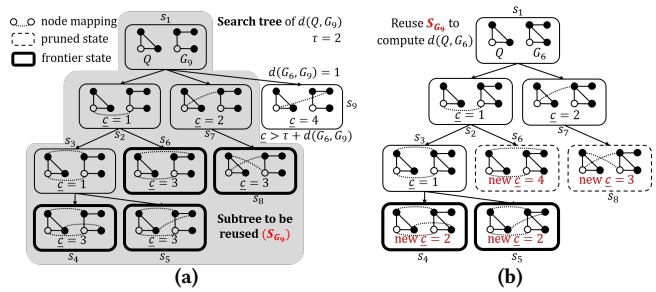


Figure 6: An illustration of search tree reuse using the example in Figure 3(c). Figure (a) shows the search tree constructed for $d(G_9, Q)$ and the search subtree S_{G_9} for reuse in computing $d(G_6, Q)$, where G_6 is a child of G_9 with $d(G_6, G_9) = 1$ and the threshold is set to $\tau = 2$, and Figure (b) shows how S_{G_9} is reused to compute $d(G_6, Q)$. The state with $\underline{c} = 4$ is pruned because $\underline{c} > \tau + d(G_6, G_9)$.

G_6, which is a child of G_9 with $d(G_6, G_9) = 1$, and $\tau = 2$. Line 3 computes the frontiers of S_{G_9} (the thick boxes). We note that s_9, having $\underline{c} = 4$, is not included in S_{G_9} because $\underline{c} > \tau + d(G_6, G_9) = 3$.

Figure 6(b) shows how S_{G_9} is used to compute $d(G_6, Q)$. In Lines 4–5, the search resumes computing $\underline{c}$ at each frontier state. The updated $\underline{c}$s are shown in red in Figure 6(b). In Line 6, s_6 is pruned because $\underline{c}(s_6) = 4 > c_{\min} = 2$. s_4, s_5, and s_8 will be further checked (Lines 6–7). The $\underline{c}$s of s_4 and s_5 are smaller than $c_{\min}$ (Line 8), and s_4 and s_5 correspond to a full mapping from G_6 to Q (Lines 10–11), the minimum edit cost is updated to 2. Also, this indicates that G_6 is an answer within the threshold τ. The search continues to check other states (Line 13). In Line 8, since $\underline{c}(s_8) = 3 > c_{\min} = 2$, s_8 is not examined further.

Analysis of EPF **query time complexity.** The query on a EPT (Algorithm 3) performs a DFS traversal, visiting each node at most once. At each visited node, the algorithm performs at most one GED computation. By Lemma 10.12 (in Appendix), the total number of nodes in a EPT is $O(|C_p|)$, where $|C_p|$ is the number of data graphs it indexes. Therefore, the worst-case NDC is $O(|C_p|)$, the

same as a linear scan of the data graphs. Among the three optimizations, subtree pruning (Section 5.1.1) checks the precomputed $\bar{d}_G$ in $O(1)$ per node, and lower-bound propagation (Section 5.1.2) computes lower bounds using the precomputed $d(G_p, G_c)$ in $O(1)$ per node; neither increases the worst-case time complexity. Search tree reuse (Section 5.1.3) does not affect the NDC, but reduces the per-computation cost. By Lemma 10.18 (in Appendix), the number of EPTs that may contain answers is at most a constant. Summing over all EPTs searched during a query, the total NDC of EPF is $O(\sum_j |C_{p_j}|)$, where p_j are the centers identified by NetDAG.

6 COST MODEL FOR Gisma CONSTRUCTION

The query efficiency bottleneck of graph similarity search in Gisma is GED computation. The radius threshold α between NetDAG and EditPathForest introduces a tradeoff in NDC between the two stages. In this section, we propose a cost model to find the optimal α that minimizes the total NDC during query processing.

The NDC of querying NetDAG can be estimated as follows. At each layer i, the children's $(\alpha \cdot 2^i)$-balls jointly cover $B(\text{pivot}, \alpha \cdot 2^{i+1} + 2\tau)$ (Lemma 4.1), which requires $\text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1} + 2\tau)$ children, where $\text{Cover}(r, R)$ denotes the minimum number of r-balls needed to cover a ball of radius R. In the worst case, the algorithm computes the distance from Q to every child. The total NDC incurred in NetDAG (including the NDC to traverse from NetDAG to EPF) is $\sum_{i=0}^{\phi-1} \text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1} + 2\tau)$.

After querying NetDAG, the candidate balls are $B(p_j^0, \alpha)$ with centers $p_j^0 \in Y_0$ satisfying $d(p_j^0, Q) \leq \alpha + \tau$ (Theorem 4.4). By the triangle inequality, all graphs g in these balls satisfy $d(g, Q) \leq d(g, p_j^0) + d(p_j^0, Q) \leq \alpha + (\alpha + \tau) = 2\alpha + \tau$. In the worst case, the NDC incurred in EPF is bounded by $|B(Q, 2\alpha + \tau)|$, the number of data graphs within distance $2\alpha + \tau$ of Q.

We combine the NDC of NetDAG and EPF into a cost model (Equation (1)) to find the optimal α.

$$\text{NDC}(\alpha) = \sum_{i=0}^{\phi-1} \text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1} + 2\tau) \ + \ |B(2\alpha + \tau)| \tag{1}$$

Since $\text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1} + 2\tau)$ cannot be directly computed, we approximate it as follows. Since τ is small relative to $\alpha \cdot 2^{i+1}$, we scale $\text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1})$ proportionally to the ball sizes. Note that $\text{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1}) = \max_p \lambda(p, \alpha \cdot 2^i)$, giving:

$$\text{NDC}(\alpha) \approx \sum_{i=0}^{\phi-1} \frac{|B(\alpha \cdot 2^{i+1} + 2\tau)|}{|B(\alpha \cdot 2^{i+1})|} \max_p \lambda(p, \alpha \cdot 2^i) + |B(2\alpha + \tau)|. \tag{2}$$

When α is small, the first term contains large summands such as $\max_p \lambda(p, \alpha)$ and $\max_p \lambda(p, 2\alpha)$, which are large due to the high doubling constant at small radii (as observed in Section 3). When α is large, $|B(2\alpha + \tau)|$ approaches the entire database size, making the second term dominant. An optimal α minimizes $\text{NDC}(\alpha)$.

Example 6.1. Based on Equation (2), Figures 4 and 7(a), we plotted Figure 7(b) to illustrate how the NDC varies with α under different τ values on the AIDS dataset. The results show that when $\tau = 2$, the minimum NDC occurs at $\alpha = 4$, while for larger thresholds ($\tau = 4$–10), the optimal α is between 9 and 12. In particular, both $\tau = 8$ and $\tau = 10$ reach their minima of α at around 12. Therefore, in our experiments, we set $\alpha = 12$ for the AIDS dataset.

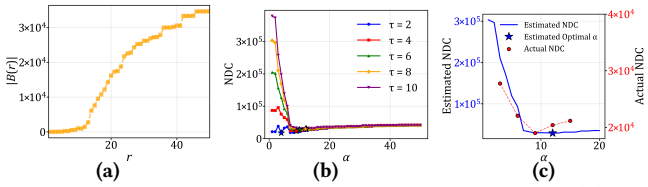


Figure 7: Illustrations of $|B(r)|$ and the estimated NDC(α) under different α values on the AIDS dataset, including the estimated curve under $\tau = 8$ and measured NDC at selected α.

Table 3: Statistics of datasets for experiments

Dataset	$\lvert\mathcal{D}\rvert$	Avg. $\lvert V\rvert$	Avg. $\lvert E\rvert$	# v_label	Used in
AIDS	42.7K	25.6	27.5	62	[7, 8, 27, 34, 36, 41]
PubChem	22.8K	48.1	50.6	10	[7, 27, 34]
Chemical1M	1M	24.0	25.8	94	-
SYN	1M	14.3	20.5	1	[7, 34]

7 EXPERIMENTAL EVALUATION

In this section, we present experiments to investigate the performance of Gisma. The code is available on Github. [3] We present the experimental settings in Section 7.1, report the overall performance on efficiency and effectiveness in Section 7.2, and provide detailed evaluation in Section 7.3.

7.1 Experimental Settings

Platform. We implemented Gisma in C++. We conducted experiments on a machine equipped with a dual 64-core AMD EPYC 7742 processor (2.25 GHz) and 2 TB RAM running Ubuntu OS.

Datasets. Following previous works [6, 7, 27, 34], we evaluated Gisma using both real-world and synthetic datasets. Specifically, we used two widely adopted real-world datasets, namely AIDS[4] and PubChem[5] [29], as well as a large-scale dataset, Chemical1M, which is also derived from the PubChem database. AIDS is a set of antiviral compounds, PubChem and Chemical1M are datasets of chemical compound graphs. To facilitate our scalability tests, we also generated a set of synthetic graphs (denoted as SYN) by using GraphGen[6]. Table 3 summarizes the statistics of these datasets.

Queries. We randomly selected 100 graphs from each dataset as query graphs and excluded them from the datasets. We followed [6, 7, 27] to report the average runtimes of all the queries.

Benchmark methods. We compared Gisma with the representative related methods. Although AStar-BMao [7] is designed for GED verification, its released code includes a filtering method. To ensure fair comparison with methods without filtering (*i.e.,* AppBMao and GEDHOT), we apply the same filtering method and denote them with "+".

- AStar-BMao+ [7]. AStar-BMao is the state-of-the-art (SOTA) for **exact** GED verification. It can answer graph similarity search queries by pairwise comparison of all data graphs and query graphs.
- App-BMao+ [41]. App-BMao is a recent method for computing *approximate* GED between two graphs. It adopts a similar A* search

[3] https://github.com/McKayGONG/Gisma
[4] https://cactus.nci.nih.gov/download/nci/AID2DA99.sdz
[5] https://pubchem.ncbi.nlm.nih.gov
[6] https://www.cse.cuhk.edu.hk/~jcheng/graphgen1.0.zip

framework to AStar-BMao, but its implementation introduces a limit that terminates the search early, at the cost of accuracy.

- GEDHOT+ [8]. GEDHOT is a recent ensemble method for *approximate* GED estimation, reported as the most accurate among learning-based GED methods. It takes advantage of a supervised model GEDIOT and an unsupervised model GEDGW.
- GHashing [36]. GHashing is a recent learning-based method for *approximate* graph similarity search. It employs graph neural networks to learn semantic hash codes for candidate generation.
- LAN [34]. LAN is a recent approximate k-nearest neighbor (AkNN) search method for graph databases based on proximity graphs. It proposes a learning-based initial node selection and neighbor pruning to reduce GED computations in query processing. We extend LAN to support similarity search by progressively increasing the search range.
- Nass [27]. Nass is the SOTA index-based solution for *exact* graph similarity search. During index construction, Nass precomputes pairwise GED among the graphs in the dataset within a certain GED range. During query processing, it uses the precomputed distances to iteratively reduce the candidate set by triangle inequality.
- Gisma. Gisma is our proposed index. During the construction of NetDAG, we use GREED [37] to compute approximate GED, as exact computation is too large for the many large-distance pairs involved. GREED is currently the fastest approximate GED method and achieves a small mean absolute error (MAE).

For presentation simplicity, we refer to AStar-BMao+ and Nass as the exact methods and App-BMao+, GEDHOT+, GHashing, and LAN as the approximate methods.

Parameters. Unless otherwise specified, the parameters are set as follows. For baseline methods, AStar-BMao+, Nass, GEDHOT+, GHashing, and LAN use their original default configurations, while App-BMao+ and Gisma are configured to achieve approximately 90% recall at each τ. The default value of α is 12 for AIDS, PubChem, and Chemical1M, and 6 for SYN, respectively.

7.2 Overall Performance

EXP-1. Overall evaluation on efficiency. We first compare the efficiency of Gisma with the benchmark methods. Table 4 presents the overall runtime performance across four benchmark datasets with various threshold values τ. The results demonstrate that Gisma is the most efficient method when $\tau \geq 6$.

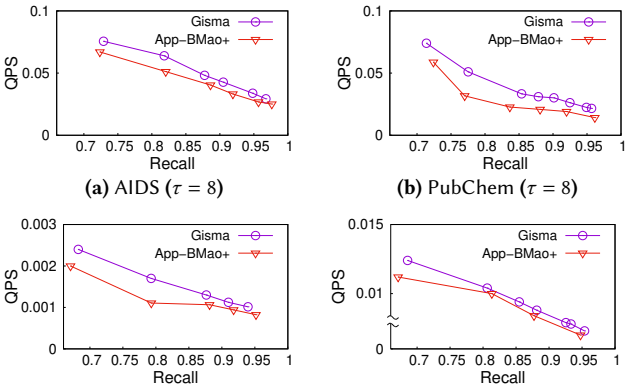
Across all datasets, while Gisma is competitive at small τ values, it is generally the most efficient method as τ increases. In addition, on AIDS and PubChem datasets, Gisma reduces the runtime by 26.6%–35.9% compared to the second-best App-BMao+ at $\tau = 12$, and shows even greater improvements (62.7%–67.9%) over AStar-BMao+. This is particularly evident in Chemical1M, where the efficiency improves from 30.8% at $\tau = 4$ to 56.3% at $\tau = 10$ when compared to the exact methods. On the SYN dataset, Gisma maintains a consistent advantage over all methods at high thresholds.

Gisma continues to be efficient across all datasets when the τ values are large. In contrast, Nass fails to build indices on the AIDS, PubChem, and SYN datasets when threshold τ is large, and it cannot build indices on Chemical1M. LAN and GHashing often cannot finish within 24 hours for large τ values. In particular, LAN performs full GED computation (without threshold-based pruning) at every routing step, which is costly when the visited graphs are far from the query. GEDHOT+ produces low-quality results across all settings, indicating that even the most accurate learning-based GED estimation method cannot reliably serve as a verifier for the search.

Table 4: Runtime comparison of benchmarked methods under default settings with varying τ (seconds)

Dataset	Method	Runtime (s)					
AIDS		τ=2	4	6	8	10	12
	Gisma	0.0070	0.1041	**1.405**	**20.41**	**259.9**	**2291**
	App-BMao+	0.0046	**0.1019**	1.752	25.04	278.2	3120
	Nass	**0.0040**	0.189	6.771	172.4	CNB	CNB
	AStar-BMao+	0.0048	0.1132	2.190	37.81	463.6	6140
	LAN	69.77	105.9	206.8	923.2	2906	CNF
	GHashing	0.3861	0.5626	4.257	48.66	LQ	LQ
	GEDHOT+	LQ	LQ	LQ	LQ	LQ	LQ
PubChem		τ=2	4	6	8	10	12
	Gisma	0.0214	0.2025	**2.445**	**30.48**	**323.6**	**3093**
	App-BMao+	0.0238	0.2511	3.853	44.82	426.9	4822
	Nass	**0.0040**	**0.144**	3.946	114.5	CNB	CNB
	AStar-BMao+	0.0250	0.2790	4.816	67.70	711.5	9634
	LAN	LQ	486.0	664.3	962.7	LQ	LQ
	GHashing	0.2502	0.5605	4.978	68.57	821.0	5585
	GEDHOT+	LQ	LQ	LQ	LQ	LQ	LQ
Chemical1M		τ=2	4	6	8	10	–
	Gisma	0.1920	**4.018**	**66.87**	**801.3**	**8247**	–
	App-BMao+	**0.1715**	5.225	87.76	956.7	11314	–
	Nass	CNB	CNB	CNB	CNB	CNB	–
	AStar-BMao+	0.1805	5.805	109.6	1667	18855	–
	LAN	LQ	LQ	CNF	CNF	CNF	–
	GHashing	7.797	13.04	LQ	LQ	CNF	–
	GEDHOT+	LQ	LQ	LQ	LQ	LQ	–
SYN		τ=1	2	3	4	5	6
	Gisma	0.8149	**7.237**	**35.15**	**129.2**	**455.2**	**1259**
	App-BMao+	0.8201	7.305	40.90	142.8	552.3	1416
	Nass	**0.7340**	14.50	CNB	CNB	CNB	CNB
	AStar-BMao+	0.8633	8.117	51.12	165.1	920.5	2933
	LAN	LQ	LQ	LQ	CNF	CNF	CNF
	GHashing	433.7	1654	7152	35141	CNF	CNF
	GEDHOT+	LQ	LQ	LQ	LQ	LQ	LQ

Bold: best runtime; Underline: second-best. CNB: Cannot build index; CNF: Cannot finish in 24h. LQ: Low-quality results where recall or precision is below 50% (see Table 7 in Appendix).



(a) AIDS ($\tau = 8$) **(b)** PubChem ($\tau = 8$) **(c)** Chemical1M ($\tau = 8$) **(d)** SYN ($\tau = 4$)

Figure 8: Comparison of QPS-recall curves

EXP-2. Overall evaluation on effectiveness. Figure 8 compares the QPS-recall trade-off between Gisma and its best competitor App-BMao+. Across all four datasets, Gisma consistently achieves higher QPS than App-BMao+ at every recall level, especially on AIDS and PubChem where Gisma achieves 1.2–1.6× higher QPS. The QPS-recall comparison with all other methods is provided in Figure 13 (appendix).

Table 5: Index sizes (MB) of different methods on the datasets, with raw dataset sizes for reference

Method	AIDS	PubChem	Chemical1M	SYN
Gisma	91.6	53.5	1797.6	1451.8
LAN	49.1	45.9	1099.6	768.9
Nass	28.1	0.8	CNB	CNB
Raw dataset	20.6	20.8	454.7	309.6

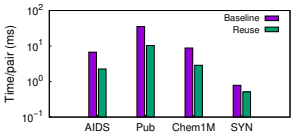


Figure 9: Effectiveness of search tree reuse in Gisma**, showing 1.54×–3.41× speedup across four datasets.**

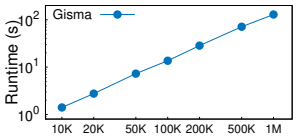


Figure 10: Scalability of Gisma **on** SYN **(τ = 4), with database size N varying from 10K to 1M.**

7.3 Detailed Evaluation of Gisma

In this section, we conduct an in-depth investigation of Gisma. Specifically, we report the index sizes and perform experimental analyses on its two stages, optimization techniques, parameter selection, and scalability.

EXP-3. Index size. Table 5 reports the index sizes of different methods, along with the corresponding database sizes for reference. The index sizes of Gisma and LAN are of the same order of magnitude as the corresponding raw database sizes. Nass has smaller index sizes but fails to build indices for Chemical1M and SYN (CNB) due to $O(N^2)$ precomputing cost.

EXP-4. Ablation study of Gisma**.** We conduct ablation experiments to analyze the contributions of the giant-step and small-step stages. We compare Gisma with the following three methods:

- Base: It is the original App-BMao+ method without the giant-step or small-step stages.
- Base + GS: It uses `NetDAG` to perform the giant-step stage to narrow down the search range, and then applies App-BMao+ to verify the graphs within this range.
- Base + SS: It performs small-step traversal on all `EditPathTrees` without the pruning of the giant-step search.

Table 6 presents the runtime comparison across various τ values on four datasets. The results show that both giant-step (GS) and small-step (SS) stages contribute to Gisma's efficiency. Compared with the Base method (without GS and SS), Gisma achieves the smallest runtimes in most cases, with runtime reductions of up to 35.9%, while Base is slightly faster than others for very small τs.

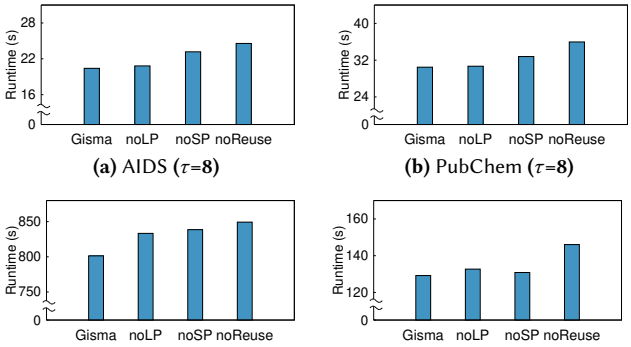
When comparing Gisma with Base + SS, Gisma achieves lower runtimes in most cases, with runtime reductions of up to 24.0% across different datasets and τs. Similarly, when comparing Gisma with Base + GS, which removes the small-step stage, the runtime reductions reach up to 33.7%. The giant-step stage is more effective for smaller τ values as it helps narrow down the search space early, while the small-step stage is increasingly effective with larger τ values. Specifically, as τ increases, the runtime difference between Gisma and Base + GS widens significantly (*e.g.*, up to 25.1% at τ=12 on AIDS).

EXP-5. Effectiveness of search tree reuse. We conduct a study on the effectiveness of search tree reuse presented in Section 5.1.3. *With Reuse* measures the average running time of single-pair GED computations with search tree reuse during the query processing of Gisma under the default setting. *Baseline* measures the average running time of the original App-BMao+ without reuse on the same graph pairs for which reuse is triggered. Figure 9 presents the average time per GED computation. Our reuse strategy achieves speedups of 1.54× to 3.41× (AIDS: 2.97×, PubChem: 3.41×, Chemical1M: 3.06×, SYN: 1.54×), reducing average runtime from 12.90ms to 4.00ms on all datasets. The speedup verifies that the proposed method of search tree reuse is effective.

Table 6: Ablation study of Gisma **(runtime in seconds)**

Dataset	Method	Runtime (s)					
AIDS		τ=2	4	6	8	10	12
	Gisma	0.0070	0.1041	**1.405**	**20.41**	**259.9**	**2291**
	Base + SS	0.0062	0.1062	1.445	21.30	264.2	2357
	Base + GS	0.0059	0.1048	1.599	23.38	276.3	3059
	Base	**0.0046**	**0.1019**	1.752	25.04	278.2	3120
PubChem		τ=2	4	6	8	10	12
	Gisma	**0.0214**	**0.2025**	**2.445**	**30.48**	**323.6**	**3093**
	Base + SS	0.0234	0.2398	3.217	34.74	348.4	3205
	Base + GS	0.0221	0.2433	3.672	38.69	404.5	4664
	Base	0.0238	0.2511	3.853	44.82	426.9	4822
Chemical1M		τ=2	4	6	8	10	–
	Gisma	0.1920	**4.018**	**66.87**	**801.3**	**8247**	–
	Base + SS	0.1944	4.372	72.76	837.2	8765	–
	Base + GS	0.2052	4.559	80.24	950.1	10874	–
	Base	**0.1715**	5.225	87.76	956.7	11314	–
SYN		τ=1	2	3	4	5	6
	Gisma	**0.8149**	**7.237**	**35.15**	**129.2**	**455.2**	**1259**
	Base + SS	0.8179	7.271	37.66	130.6	485.8	1311
	Base + GS	0.8162	7.254	38.67	140.3	527.4	1380
	Base	0.8201	7.305	40.90	142.8	552.3	1416

Bold indicates the best (lowest) runtime; underline indicates the second-best one.

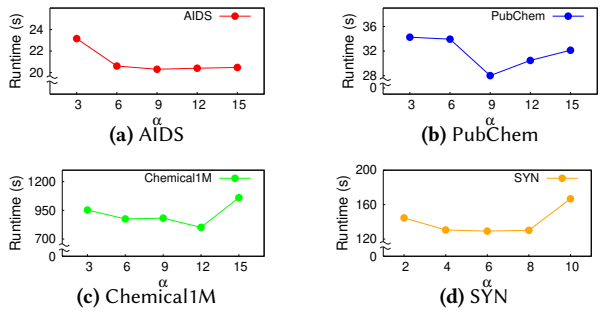


(a) AIDS **(τ=8)** **(b)** PubChem **(τ=8)** **(c)** Chemical1M **(τ=8)** **(d)** SYN **(τ=4)**

Figure 11: Ablation study of `EditPathForest` **optimizations**

EXP-6. Ablation study of `EditPathForest` **optimizations.** Figure 11 evaluates the three optimizations for EPT query. We compare Gisma with three variants: noReuse (no search tree reuse), noSP (no subtree pruning), and noLP (no lower-bound propagation), each of which removes one optimization from Gisma. The results show that all three optimizations contribute to Gisma's efficiency. Across all datasets, search tree reuse has the largest impact, followed by subtree pruning and lower-bound propagation.

EXP-7. Effects of α. α is the radius threshold between `NetDAG` and `EditPathForest` in Gisma. Figure 12 evaluates the impact of



(a) AIDS **(b)** PubChem **(c)** Chemical1M **(d)** SYN

Figure 12: Effects of radius threshold α on runtime performance

different α values on Gisma's performance. For AIDS, PubChem, and Chemical1M, we test $\alpha \in \{3, 6, 9, 12, 15\}$, while for SYN we use $\alpha \in \{2, 4, 6, 8, 10\}$. We also verify that we can use the cost model presented in Section 6 to determine an optimal α (Figure 7(c)).

EXP-8. Scalability. We evaluate the scalability of Gisma by varying the database size N on the SYN dataset with $\tau = 4$. Figure 10 shows the runtime as N increases from 10K to 1M. The runtime grows linearly with N, confirming the scalability of Gisma.

8 RELATED WORK

Similarity search in doubling space. Several studies [4, 26, 30] observed that similarity search queries can be answered more efficiently in metric spaces with bounded intrinsic dimensionality. Specifically, Karger et al. [26] introduced the *expansion constant* c to measure the growth of metric spaces. In a metric space, for any point p in the space and any radius r, a ratio is defined between the number of points within a ball of radius $2r$ and that within a ball of radius r, both centered at p. The maximum of this ratio over all p and r is defined as the *expansion constant* c of the space. Based on this constant, a sampling-based index for nearest neighbor search was designed in [26]. Similar to [26], Krauthgamer and Lee [30] proposed the *doubling constant* λ, which is defined as the minimum number of balls having radius r by which a ball with radius $2r$ can be covered. Then, they proposed the Navigating-nets index, which guarantees $(1+\epsilon)$-approximate nearest neighbor searches. Beygelzimer et al. [4] proposed cover-tree, achieving search complexity $O(c^{12} \log N)$, where N is the dataset size. Izbicki and Shelton [24] proposed parallel algorithms for constructing and searching in cover trees. Recently, Elkin and Kurlin [14] proposed a compressed version of the cover tree, providing a rigorous complexity analysis. Another hierarchical structure, the Net-tree [22], organizes data into hierarchical subsets (r-nets) using the doubling constant to efficiently answer queries. Some studies observed that the doubling property of metric spaces can vary substantially across different regions or scales, and proposed corresponding notions and structures such as *nearly doubling spaces* [19] and the *t-restricted doubling dimension* with the *net-forest* index [9].

Exact graph similarity search. The *filtering-and-verification* framework [6, 7, 27, 28, 31, 40, 44, 45] is widely adopted for graph similarity search. Here, we briefly review a few recent representative GED verification methods (*i.e.,* determining whether GED $\leq \tau$). Riesen et al. [38] proposed a practical GED computation method *A*GED* by adopting the best-first search strategy. To improve A* search for GED verification, Chang et al. [6] proposed AStar-LSa, which reduces search space by discarding dummy vertices, rearranging matching order, and providing a tighter lower bound (LSa). They subsequently introduced a tighter lower bound (BMao) in AStar-BMao [7], which is the SOTA method for GED verification. Gouda and Hassaan [20] proposed CSI-GED, which reformulates GED as common sub-structure isomorphism enumeration and also supports threshold-based verification. Kim [28] proposed Inves, an A* search method with a partition-based lower bound, and Nass [27], the SOTA for index-based GED search. However, Nass is costly to construct as its index requires $O(N^2)$ GED computations. Recently, exact GED computation methods such as DF-GED [1] (depth-first search) and the ILP-based method [12] have been proposed, but they do not support GED verification.

Approximate graph similarity search. Several studies address approximate graph similarity search [3, 34, 36]. Qin et al. [36] proposed GHashing, which employs hash codes and continuous embeddings as machine-learning-based filters for candidate selection, and then, verified by exact GED. LAN [34] enhances a proximity graph to query graph databases, which proposes learning-based initial point selection and neighbor pruning to reduce GED computations. In addition, machine-learning-based methods [2, 35, 37, 43] are proposed to approximate GED. Specifically, SimGNN [2] employs graph convolutional networks (GCNs) and neural tensor networks for GED prediction. Noah [43] integrates the A*-beam search with a *Graph Path Network (GPN)* heuristic. GREED [37] enables a linear-time GED prediction. GEDGNN [35] jointly predicts GED values and vertex mappings. These approaches provide practical approximations when exact GED computations do not finish. Cheng et al. [8] proposed GEDHOT, an ensemble method that combines a supervised inverse optimal transport model (GEDIOT) with an unsupervised Gromov–Wasserstein–based model (GEDGW). It reports state-of-the-art accuracy for machine-learning-based approximate GED estimation. Recently, Xu et al. [41] proposed a heuristic approximate GED computation method App-BMao, which adapts the A* search framework with an iteration limit mechanism to achieve speedup with limited accuracy loss.

9 CONCLUSION

In this paper, we analyzed the GED space and identified a two-stage phenomenon: the doubling/expansion constant is small at large distances but large at small distances. Based on this insight, we proposed Gisma, a two-stage index for approximate graph similarity search. The first stage, NetDAG, exploits the doubling property to narrow the search space with $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ query complexity, the lowest among existing doubling-based methods. The second stage, EPF, indexes shared graph edit paths for fine-grained search with three optimizations. Experiments on four benchmark datasets show that Gisma outperforms six representative methods in most cases with high recall. As future work, we plan to integrate Gisma with machine learning methods to further improve query efficiency.

AI USAGE DECLARATION

Generative AI tools were used only for polishing paper writing and assisting code writing. All AI-assisted edits were carefully reviewed and finalized by authors.

REFERENCES

[1] Zeina Abu-Aisheh, Romain Raveaux, Jean-Yves Ramel, and Patrick Martineau. 2015. An Exact Graph Edit Distance Algorithm for Solving Pattern Recognition Problems. In *Proc. of the International Conference on Pattern Recognition Applications and Methods*. 271–278.

[2] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. 2019. Simgnn: A neural network approach to fast graph similarity computation. In *Proceedings of the twelfth ACM international conference on web search and data mining*. 384–392.

[3] Yunsheng Bai, Hao Ding, Ken Gu, Yizhou Sun, and Wei Wang. 2020. Learning-Based Efficient Graph Similarity Computation via Multi-Scale Convolutional Set Matching.. In *Proc. of AAAI Conference on Artificial Intelligence (AAAI)*. 3219–3226.

[4] Alina Beygelzimer, Sham M. Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the International Conference on Machine Learning (ICML)*, Vol. 148. 97–104.

[5] Silvio Cesare and Yang Xiang. 2011. Malware Variant Detection Using Similarity Search over Sets of Control Flow Graphs. In *Proc. of the International Conference on Trust, Security and Privacy in Computing and Communications*. 181–189.

[6] Lijun Chang, Xing Feng, Xuemin Lin, Lu Qin, Wenjie Zhang, and Dian Ouyang. 2020. Speeding Up GED Verification for Graph Similarity Search. In *36th IEEE International Conference on Data Engineering (ICDE)*. 793–804.

[7] Lijun Chang, Xing Feng, Kai Yao, Lu Qin, and Wenjie Zhang. 2022. Accelerating Graph Similarity Search via Efficient GED Computation. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4485–4498.

[8] Qihao Cheng, Da Yan, Tianhao Wu, Zhongyi Huang, and Qin Zhang. 2025. Computing approximate graph edit distance via optimal transport. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.

[9] Aruni Choudhary and Michael Kerber. 2015. Local Doubling Dimension of Point Sets. In *Proceedings of the 27th Canadian Conference on Computational Geometry (CCCG)*. Kingston, Ontario, Canada, 156–164.

[10] Kenneth L. Clarkson. 2006. Nearest-Neighbor Searching and Metric Space Dimensions. In *Nearest-Neighbor Methods for Learning and Vision: Theory and Practice*, Gregory Shakhnarovich, Trevor Darrell, and Piotr Indyk (Eds.). MIT Press.

[11] Aurelio Antelo Collado, Ramón Carrasco-Velar, Nicolás García-Pedrajas, and Gonzalo Cerruela-García. 2020. Maximum Common Property: A New Approach for Molecular Similarity. *Journal of Cheminformatics* 12, 61 (2020).

[12] Andrea D'Ascenzo, Julian Meffert, Petra Mutzel, and Fabrizio Rossi. 2025. Enhancing Graph Edit Distance Computation: Stronger and Orientation-Based ILP Formulations. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4737–4749. https://doi.org/10.14778/3749646.3749726

[13] Hui Du. 2010. Chemical Molecules Search Based on Graph Similarity Measure. In *Proc. of the International Conference on Biomedical Engineering and Computer Science*. 1–4.

[14] Yury Elkin and Vitaliy Kurlin. 2023. A new near-linear time algorithm for k-nearest neighbor search using a compressed cover tree. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 202. 9267–9311.

[15] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[16] Carlos Garcia-Hernandez, Alberto Fernández, and Francesc Serratosa. 2019. Ligand-Based Virtual Screening Using Graph Edit Distance as Molecular Similarity Measure. *Journal of Chemical Information and Modeling* 59, 4 (2019), 1410–1421.

[17] Yikai Gong, Lyu Xu, Byron Choi, Jianliang Xu, and Xin Huang. 2026. Gisma: A Giant-Step-Small-Step Indexing Framework for Approximate Similarity Search in Graph Databases (Technical Report). https://github.com/McKayGONG/Gisma.

[18] Lee-Ad Gottlieb, Aryeh Kontorovich, and Robert Krauthgamer. 2014. Efficient classification for metric data. *IEEE Transactions on Information Theory* 60, 9 (2014), 5750–5759.

[19] Lee-Ad Gottlieb and Robert Krauthgamer. 2013. Proximity algorithms for nearly doubling spaces. *SIAM Journal on Discrete Mathematics* 27, 4 (2013), 1759–1769.

[20] Karam Gouda and Mosab Hassaan. 2016. CSI_GED: An efficient approach for graph edit similarity computation. In *Proc. of the IEEE International Conference on Data Engineering (ICDE)*. 265–276.

[21] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 171–186.

[22] Sariel Har-Peled and Manor Mendel. 2005. Fast Construction of Nets in Low Dimensional Metrics, and Their Applications. In *Proceedings of the Twenty-First Annual Symposium on Computational Geometry* (Pisa, Italy) *(SCG '05)*. Association for Computing Machinery, New York, NY, USA, 150–158. https://doi.org/10.1145/1064092.1064117

[23] Wassily Hoeffding. 1992. A class of statistics with asymptotically normal distribution. In *Breakthroughs in statistics: Foundations and basic theory*. Springer, 308–334.

[24] Mike Izbicki and Christian Shelton. 2015. Faster cover trees. In *International Conference on Machine Learning*. PMLR, 1162–1170.

[25] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2025. Fast Graph-Based Vector Search via Hardware-Accelerated Delayed-Synchronization Traversal. *Proceedings of the VLDB Endowment* 18, 14 (2025), 3797–3811.

[26] David R Karger and Matthias Ruhl. 2002. Finding nearest neighbors in growth-restricted metrics. In *Proceedings of the thiry-fourth annual ACM symposium on Theory of computing*. 741–750.

[27] Jongik Kim. 2021. Boosting Graph Similarity Search through Pre-Computation. In *Proc. of the ACM SIGMOD International Conference on Management of Data*. 951–963.

[28] Jongik Kim, Dong-Hoon Choi, and Chen Li. 2019. Inves: Incremental Partitioning-Based Verification for Graph Similarity Search.. In *EDBT*. 229–240.

[29] Sunghwan Kim, Jie Chen, Tiejun Cheng, Asta Gindulyte, Jia He, Siqian He, Qingliang Li, Benjamin A Shoemaker, Paul A Thiessen, Bo Yu, et al. 2023. PubChem 2023 update. *Nucleic acids research* 51, D1 (2023), D1373–D1380.

[30] Robert Krauthgamer and James R. Lee. 2004. Navigating nets: simple algorithms for proximity search. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, 2004*, J. Ian Munro (Ed.). SIAM, 798–807.

[31] Yongjiang Liang and Peixiang Zhao. 2017. Similarity search in graph databases: A multi-layered indexing approach. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 783–794.

[32] Y. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[33] Misagh Naderi, Chris Alvin, Yun Ding, Supratik Mukhopadhyay, and Michal Brylinski. 2016. A graph-based approach to construct target-focused libraries for virtual screening. *Journal of cheminformatics* 8, 1 (2016), 1–16.

[34] Yun Peng, Byron Choi, Tsz Nam Chan, and Jianliang Xu. 2022. Lan: Learning-based approximate k-nearest neighbor search in graph databases. In *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 2508–2521.

[35] Chengzhi Piao, Tingyang Xu, Xiangguo Sun, Yu Rong, Kangfei Zhao, and Hong Cheng. 2023. Computing Graph Edit Distance via Neural Graph Matching. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1817–1829.

[36] Zongyue Qin, Yunsheng Bai, and Yizhou Sun. 2020. GHashing: Semantic Graph Hashing for Approximate Similarity Search in Graph Databases. In *Proc. of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2062–2072.

[37] Rishabh Ranjan, Siddharth Grover, Sourav Medya, Venkatesan Chakaravarthy, Yogish Sabharwal, and Sayan Ranu. 2022. Greed: A neural framework for learning graph distance functions. *Advances in Neural Information Processing Systems* 35 (2022), 22518–22530.

[38] Kaspar Riesen, Stefan Fankhauser, and Horst Bunke. 2007. Speeding up graph edit distance computation with a bipartite heuristic.. In *MLG*. Citeseer, 21–24.

[39] Runzhong Wang, Tianqi Zhang, Tianshu Yu, Junchi Yan, and Xiaokang Yang. 2021. Combinatorial learning of graph edit distance via dynamic embedding. In *Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5241–5250.

[40] Xiaoli Wang, Xiaofeng Ding, Anthony K.H. Tung, Shanshan Ying, and Hai Jin. 2012. An efficient graph indexing method. In *Proc. of the IEEE International Conference on Data Engineering*. IEEE, 210–221.

[41] Mouyi Xu and Lijun Chang. 2025. Graph Edit Distance Estimation: A New Heuristic and A Holistic Evaluation of Learning-based Methods. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.

[42] Xiaoliang Xu, Mengzhao Wang, Yuxiang Wang, and Dingcheng Ma. 2021. Two-stage routing with optimized guided search and greedy algorithm on proximity graph. *Knowledge-Based Systems* 229 (2021), 107305. https://doi.org/10.1016/j.knosys.2021.107305

[43] Lei Yang and Lei Zou. 2021. Noah: Neural-optimized A* Search Algorithm for Graph Edit Distance Computation. In *Proc. of IEEE International Conference on Data Engineering (ICDE)*. 576–587.

[44] Zhiping Zeng, Anthony K. H. Tung, Jianyong Wang, Jianhua Feng, and Lizhu Zhou. 2009. Comparing Stars: On Approximating Graph Edit Distance. *Proc. VLDB Endow.* 2, 1 (2009), 25–36.

[45] Xiang Zhao, Chuan Xiao, Xuemin Lin, Wei Wang, and Yoshiharu Ishikawa. 2013. Efficient processing of graph similarity queries with edit distance constraints. *The VLDB Journal* 22, 6 (2013), 727–752.

[46] Xiang Zhao, Chuan Xiao, Xuemin Lin, Wenjie Zhang, and Yang Wang. 2018. Efficient structure similarity searches: a partition-based approach. *The VLDB Journal* 27, 1 (2018), 53–78.

[47] Yuanyuan Zhu, Lu Qin, Jeffrey Xu Yu, and Hong Cheng. 2020. Answering Top-k Graph Similarity Queries in Graph Databases. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2020), 1459–1474.

10 APPENDIX

10.1 Supplementary Analysis of NetDAG

This subsection establishes the space, query time, and construction complexity of NetDAG. We adopt a lemma from [18] (Lemma 10.1) to bound the number of children and parents per node (Lemmas 10.2–10.3) and the number of inter-layer edges per pair (Lemma 10.4). These lead to the main complexity results: space (Theorem 10.5), query time (Theorem 10.6), and construction time (Theorem 10.7). We also express the bounds in terms of the expansion constant c (Lemma 10.8) for comparison with Cover Tree.

Lemma 10.1 ([18]). *Let (X, d) be a metric space with doubling constant λ. Suppose $S \subset X$ is finite, with minimum inter-point distance at least $r > 0$ and diameter at most* $\mathrm{diam} > 0$. *Then*

$$|S| \;\le\; \lambda^{\log_2\left(\frac{2\,\mathrm{diam}}{r}\right)}.$$

Corollary 2 (Number of r-net points in a ball)**.** Given an r-net Y in a metric space (X, d) with doubling constant λ, for any ball B of radius at most R, the number of points of Y contained in B is at most

$$\lambda^{\log_2\left(\frac{4R}{r}\right)}.$$

Proof of Corollary 2. Let $Y' = Y \cap B$. Because Y' is contained in a ball of radius R, the diameter of Y' is at most $2R$. Meanwhile, Y being an r-net implies any two distinct points in Y' are at least r apart. Hence Y' meets the conditions of Lemma 10.1 with minimum inter-point distance r and diameter at most $2R$. Applying Lemma 10.1 yields

$$|Y'| \;=\; |\,Y \cap B| \;\le\; \lambda^{\log_2\left(\frac{2(2R)}{r}\right)} \;=\; \lambda^{\log_2\left(\frac{4R}{r}\right)}.$$

□

Lemma 10.2 (Upper bound on number of children). *In an* NetDAG *index (Definition 4.1) with restricted doubling constant λ_α, where α is the ball radius of centers in layer Y_0 and $0 \le \tau \le \alpha$, the number of children of any node is at most*

$$\lambda_\alpha^{\log_2 20}.$$

Proof. To obtain this constant, we analyze the child range and separation of NetDAG. From Definition 4.1, each child p_x^i of p_j^{i+1} satisfies $d\big(p_j^{i+1},\ p_x^i\big) \le 3\,\alpha\,2^i + 2\,\tau$, so all children are in the ball $B\big(p_j^{i+1},\ 3\,\alpha\,2^i + 2\,\tau\big)$. Since Y_i is an $(\alpha\,2^i)$-net, the minimal distance between any two distinct nodes in Y_i is at least $\alpha\,2^i$. Applying Corollary 2 with $R = 3\,\alpha\,2^i + 2\,\tau$ and $r = \alpha\,2^i$ gives an upper bound of

$$\lambda^{\log_2\left(\frac{4\,(3\,\alpha\,2^i+2\,\tau)}{\alpha\,2^i}\right)} = \lambda^{\log_2\left(12+\frac{8\,\tau}{\alpha\,2^i}\right)}.$$

Since $0 \le \tau \le \alpha$ and children are always in a layer Y_i with $i \ge 0$, we have $12 + \frac{8\tau}{\alpha 2^i} \le 12 + \frac{8\alpha}{\alpha 2^i} \le 20$. Replacing the doubling constant by λ_α for the relevant scale yields the claimed bound $\lambda_\alpha^{\log_2 20}$. □

Lemma 10.3 (Upper bound on number of parents). *In an* NetDAG *index (Definition 4.1) with restricted doubling constant λ_α, where α is the ball radius of centers in layer Y_0 and $0 \le \tau \le \alpha$, the number of parents of any node p_x^i is at most*

$$\lambda_\alpha^{\log_2 10}.$$

Proof. A node p_x^i in layer Y_i can only have parents in layer Y_{i+1} that satisfy $d\big(p_x^i,\ p_j^{i+1}\big) \le 3\,\alpha\,2^i + 2\,\tau$, so all possible parents are in $B\big(p_x^i,\ 3\,\alpha\,2^i + 2\,\tau\big) \cap Y_{i+1}$. Since Y_{i+1} is an $(\alpha\,2^{i+1})$-net, any two distinct points there are at least $\alpha\,2^{i+1}$ apart. Applying Corollary 2 with $R = 3\,\alpha\,2^i + 2\,\tau$ and $r = \alpha\,2^{i+1}$ gives an upper bound of

$$\lambda^{\log_2\left(\frac{4\,(3\,\alpha\,2^i+2\,\tau)}{\alpha\,2^{i+1}}\right)} = \lambda^{\log_2\left(6+\frac{4\,\tau}{\alpha\,2^i}\right)}.$$

Since $0 \le \tau \le \alpha$ and $i \ge 0$, we have $6 + \frac{4\tau}{\alpha 2^i} \le 6 + \frac{4\alpha}{\alpha\cdot 1} = 10$. Replacing the doubling constant by λ_α for the relevant scale yields the claimed bound $\lambda_\alpha^{\log_2 10}$. □

In a NetDAG, the same data point p_k may appear as a child of p_j in multiple consecutive layers. The following lemma shows that the number of such layers is bounded by a constant, which is used in the space complexity proof (Theorem 10.5).

For each data point p_x, let $i_x^{\max}$ be such that $Y_{i_x^{\max}}$ is the highest layer containing p_x (*i.e.*, $p_x \in Y_{i_x^{\max}}$ but $p_x \notin Y_{i_x^{\max}+1}$).

Lemma 10.4. *For any pair of data points (p_j, p_k) in an* NetDAG *index, if the parent-child edge $(p_j^{i_0+1}, p_k^{i_0})$ exists at some layer i_0, then:* ***(a)*** *the edge exists at all higher layers as long as p_j and p_k exist: the edge (p_j^{i+1}, p_k^i) exists at all layers i with $i_0 \le i \le \min(i_j^{\max}-1,\ i_k^{\max})$;* ***(b)*** *the number of such layers is at most a constant:* $\min(i_j^{\max} - 1,\ i_k^{\max}) - i_0 + 1 \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor + 1$.

Proof of Lemma 10.4. By Definition 4.1, the edge (p_j^{i+1}, p_k^i) exists at layer i if and only if $d(p_j, p_k) \le 3 \cdot \alpha \cdot 2^i + 2\tau$, $p_j \in Y_{i+1}$, and $p_k \in Y_i$.

(a) Since $(p_j^{i_0+1}, p_k^{i_0})$ exists, we have $d(p_j, p_k) \le 3\alpha \cdot 2^{i_0} + 2\tau$. For all $i \ge i_0$, $3\alpha \cdot 2^{i_0} + 2\tau \le 3\alpha \cdot 2^i + 2\tau$, so the distance condition is satisfied. The edge also requires $p_k \in Y_i$ ($i \le i_k^{\max}$) and $p_j \in Y_{i+1}$ ($i \le i_j^{\max} - 1$). Hence the edge exists at all layers i with $i_0 \le i \le \min(i_j^{\max} - 1,\ i_k^{\max})$.

(b) By (a), the edge $(p_j^{i_0+m+1}, p_k^{i_0+m})$ exists, where $m = \min(i_j^{\max} - 1,\ i_k^{\max}) - i_0$. Since $Y_{i_0+m+1} \subseteq Y_{i_0+m}$, both $p_j, p_k \in Y_{i_0+m}$. Then by the $(\alpha \cdot 2^{i_0+m})$-net property (Definition 2.4), $d(p_j, p_k) \ge \alpha \cdot 2^{i_0+m}$. Combined with $d(p_j, p_k) \le 3\alpha \cdot 2^{i_0} + 2\tau$ since the edge $(p_j^{i_0+1}, p_k^{i_0})$ exists:

$$\alpha \cdot 2^{i_0+m} \le d(p_j, p_k) \le 3 \cdot \alpha \cdot 2^{i_0} + 2\tau.$$

Dividing by $\alpha \cdot 2^{i_0}$ gives $2^m \le 3 + 2\,\tau/(\alpha \cdot 2^{i_0}) \le 3 + 2\,\tau/\alpha$ (since $i_0 \ge 0$), so $m \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor$, and therefore $m + 1 = \min(i_j^{\max} - 1,\ i_k^{\max}) - i_0 + 1 \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor + 1$. □

It should be remarked that the space complexity is $O(N)$. The proof requires a non-trivial extension of the analysis in [30]. Due to space limitations, we present the proof in Theorem 10.5.

Theorem 10.5 (Space Complexity of NetDAG). *The total space of* NetDAG *is $O(\lambda_\alpha^{O(1)} N) = O(N)$, where N is the database size and λ_α is the restricted doubling constant, with α being the ball radius of centers in layer Y_0.*

Proof. The space of NetDAG equals the number of nodes and edges. By Lemma 10.2, each node has at most $\lambda_\alpha^{\log_2 20}$ children. It therefore suffices to show that the total number of nodes is $O(N)$.

Following [30], we only count nodes with multiple children. By Definition 4.1, $Y_i \subseteq Y_{i-1}$, every node p_j^i is by default its own child p_j^{i-1} in the next layer. Thus a node with exactly one child need not be stored and does not contribute to NetDAG space. It remains to show that the number of nodes with multiple children is $O(N)$.

For ease of analysis of counting those nodes, we define a *marking set* for each node p_j^i that has multiple children:

$$M_j^i = \left\{ p_k \;\middle|\; \Psi_1(p_j^i,\, p_k^{i-1}) \;\vee\; \Psi_2(p_k^{i+1},\, p_j^i) \right\},$$

where Ψ_1 and Ψ_2 are two *marking conditions*:
$\Psi_1(p_j^i,\, p_k^{i-1})$: the edge $(p_j^i,\, p_k^{i-1})$ exists and $i-1 \in [i_k^{\max}-m,\; i_k^{\max}]$;
$\Psi_2(p_k^{i+1},\, p_j^i)$: the edge $(p_k^{i+1},\, p_j^i)$ exists and $i \in [i_j^{\max} - m,\; i_j^{\max}]$;
and $m = \lfloor \log_2(2\,(\tau/\alpha) + 3) \rfloor$.

Since each node with multiple children has only one marking set, the number of such nodes equals the number of marking sets. We show that every M_j^i is non-empty (Step 1), so the number of marking sets is at most $\sum_{j,i} |M_j^i|$, where $j \in \{1, \ldots, N\}$, $i \in \{0, \ldots, \phi\}$, and the sum is over all nodes p_j^i with multiple children. We then show $\sum_{j,i} |M_j^i| = O(N)$ (Step 2). Together, these two steps prove that the number of nodes with multiple children is $O(N)$.

Step 1: Every marking set is non-empty. Consider an arbitrary node $p_j^{i_0}$ with multiple children, and a child $p_k^{i_0-1}$ with $k \neq j$. By the definition of $M_j^{i_0}$, it suffices to show that $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ or $\Psi_2(p_k^{i_0+1},\, p_j^{i_0})$ holds. Since the edge $(p_j^{i_0},\, p_k^{i_0-1})$ exists, by Lemma 10.4:

Case 1: $i_k^{\max} < i_j^{\max}$. By (b), $i_k^{\max} - (i_0 - 1) \leq m$, therefore $i_0 - 1 \in [i_k^{\max} - m,\; i_k^{\max}]$, so $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ holds. Hence $p_k \in M_j^{i_0}$.

Case 2: $i_k^{\max} \geq i_j^{\max}$.

- If $i_k^{\max} = i_0$: $i_0 - 1 \in [i_k^{\max} - m,\; i_k^{\max}]$ (since $m \geq 1$), so $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ holds. Hence $p_k \in M_j^{i_0}$.
- If $i_k^{\max} \geq i_0 + 1$: $p_k \in Y_{i_0+1}$, and $d(p_j, p_k) \leq 3\alpha \cdot 2^{i_0-1} + 2\tau \leq 3\alpha \cdot 2^{i_0} + 2\tau$, so the edge $(p_k^{i_0+1},\, p_j^{i_0})$ exists. By (b), $\min(i_j^{\max} - 1,\; i_k^{\max}) = i_j^{\max} - 1$, so $(i_j^{\max} - 1) - (i_0 - 1) \leq m$, *i.e.*, $i_0 \in [i_j^{\max} - m,\; i_j^{\max}]$, so $\Psi_2(p_k^{i_0+1},\, p_j^{i_0})$ holds. Hence $p_k \in M_j^{i_0}$.

In both cases, $|M_j^{i_0}| \geq 1$.

Step 2: Upper bound of the sum of sizes of all marking sets. Every element of M_j^i has a parent-child edge satisfying Ψ_1 or Ψ_2. Each such edge contributes at most 2 elements. For each data point p_b as the child in an edge, the condition $i \in [i_b^{\max} - m,\; i_b^{\max}]$ (used in both Ψ_1 and Ψ_2) holds for at most $m+1$ layers ($m+1 = \lfloor \log_2(2\,(\tau/\alpha)+3) \rfloor + 1$ is a constant), and at each such layer i, the number of parents p_a^{i+1} is $O(1)$ by Lemma 10.3. Hence each p_b is involved in $O(1)$ edges satisfying Ψ_1 or Ψ_2, and $\sum_{j,i} |M_j^i| \leq N \cdot (m+1) \cdot O(1) \cdot 2 = O(N)$.

Putting these together, since each marking set is non-empty, the number of nodes with multiple children is at most $\sum_{j,i} |M_j^i| = O(N)$. By Lemma 10.2, each node has only a constant number of children. Thus the total space of NetDAG is $O(N)$. □

Theorem 10.6 (Time Complexity of Querying NetDAG). *Given a* NetDAG *with ϕ layers, Algorithm 1 completes in $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ distance computations (NDCs).*

Proof. The algorithm traverses from layer Y_ϕ down to the leaf layer Y_1, maintaining a single pivot at each layer. At each layer i (for $i = \phi - 1$ down to 1), we examine all children of the current pivot p^{i+1} to find one satisfying

$$d(p_j^i, Q) \leq \alpha \cdot 2^i + \tau.$$

By Lemma 10.2, any node has at most

$$\lambda_\alpha^{\log_2\left(12 + \frac{8\tau}{\alpha \cdot 2^i}\right)} \;\leq\; \lambda_\alpha^{\log_2 20}$$

children. Therefore, the algorithm performs at most $\lambda_\alpha^{\log_2 20}$ distance computations per layer across ϕ layers. This yields a total of

$$O(\lambda_\alpha^{\log_2 20} \cdot \phi)$$

distance computations.

Finally, note that $\phi = O(\log_2(\Delta/\alpha))$, where Δ is the dataset diameter (the maximum pairwise distance between graphs). □

Theorem 10.7 (Construction Complexity of NetDAG). *The* NetDAG *index can be constructed using $O(\lambda_\alpha^{O(1)} N\phi)$ distance computations, where N is the database size, ϕ is the number of layers, and λ_α is the doubling constant restricted to radii $\geq \alpha$.*

Proof. Section 3.1 of [22] shows that the number of distance computations conducted during the greedy-permutation construction is $O(\lambda^{O(1)} Ns)$, where s is the number of *phases*. The radius sequence $r_1 \geq r_2 \geq \cdots$ is non-increasing, and a *phase* is the maximal interval of consecutive iterations in which the greedy radius r_k stays within $(r/2, r]$ for some value r. Within any such *phase*, each point is scanned only $\lambda^{O(1)}$ times (Section 3.1 in [22]), because the doubling property bounds the size of every *friends list* and hence, the number of times a point recomputes its distance to the newly selected center. Summed over all *phases*, the total number of distance computations is therefore $O(\lambda^{O(1)} Ns)$.

In our setting, the greedy radii decrease from $\alpha \cdot 2^\phi$ to α, and the radius changes by at most a factor of two between consecutive layers. Hence the construction encounters exactly $\phi+1$ distinct radii, which correspond directly to the $\phi+1$ layers of NetDAG. Substituting $s = \phi + 1$ gives

$$O(\lambda^{O(1)} Ns) = O(\lambda^{O(1)} N\phi).$$

Finally, all radii used in our construction satisfy $r \geq \alpha$, so all arguments in [22] remain valid when the global doubling constant λ is replaced by the restricted doubling constant $\lambda_\alpha := \max_{p \in \mathcal{X},\, r \geq \alpha} \lambda(p, r)$. Thus the total number of distance computations becomes $O(\lambda_\alpha^{O(1)} N\phi)$. □

10.1.1 *Time and space complexity in terms of expansion constant.*

The preceding theorems express all bounds using the doubling constant λ. The time and space complexity of NetDAG can also be stated in terms of the expansion constant c, which allows direct comparison with Cover Tree [4] (Table 2). We adopt and generalize a technique from Cover Trees [4] as follows.

Lemma 10.8. *Given a metric space $(\mathcal{X}, d)$ with expansion constant c (Def. 2.2) and an r-separated set $S_r \subseteq \mathcal{X}$, for any point $p \in \mathcal{X}$ and any integer $k \geq 0$, the number of points of S_r contained in the ball $B(p,\, 2^k r)$ is at most c^{k+3}.*

Proof. The proof generalizes the *width bound* (Lemma 4.1) of Cover Trees [4] to arbitrary k. (Substituting $r = 2^{i-1}$ and $k = 1$

recovers the Cover Tree bound c^4 exactly; our formulation yields the c^{k+3} bounds used in Table 2.)

We fix $p \in \mathcal{X}$ and let $S := S_r \cap B(p, 2^k r)$. Since S_r is an r-separated set, any two points of S are at least r apart. Hence, the balls $\{B(p', r/2) : p' \in S\}$ are pairwise disjoint. Moreover, for any $p' \in S$,

$$B(p', r/2) \subseteq B\big(p,\ 2^k r + r/2\big) \subseteq B\big(p,\ 2^{k+1} r\big),$$

so

$$\sum_{p' \in S} |B(p', r/2)| \ \le\ \big|B\big(p, 2^{k+1} r\big)\big|. \tag{3}$$

Let $p^* \in \arg\min_{p' \in S} |B(p', r/2)|$. Since $d(p, p^*) \le 2^k r$, for any $y \in B\big(p, 2^{k+1} r\big)$,

$$d(y, p^*) \le d(y, p) + d(p, p^*) \le 2^{k+1} r + 2^k r < 2^{k+2} r.$$

Hence,

$$B\big(p, 2^{k+1} r\big) \subset B\big(p^*, 2^{k+2} r\big).$$

Since $B(p, 2^{k+1} r) \subset B(p^*, 2^{k+2} r)$, we have $\big|B\big(p, 2^{k+1} r\big)\big| \le \big|B\big(p^*, 2^{k+2} r\big)\big|$. By the expansion inequality $|B(p, 2r)| \le c\,|B(p, r)|$ (Def. 2.2),

$$\begin{aligned} \big|B\big(p^*, 2^{k+2} r\big)\big| \ &\le\ c\,\big|B\big(p^*, 2^{k+1} r\big)\big| \ \le\ c^2\,\big|B\big(p^*, 2^k r\big)\big| \\ &\le\ \cdots\ \le\ \boldsymbol{c^{k+3}}\,\big|B(p^*, r/2)\big|. \end{aligned} \tag{4}$$

Combining (3) and (4), since $|B(p^*, r/2)| \le |B(p', r/2)|$ for all $p' \in S$,

$$|S| \cdot |B(p^*, r/2)| \ \le\ \sum_{p' \in S} |B(p', r/2)| \ \le\ \boldsymbol{c^{k+3}}\,|B(p^*, r/2)|,$$

so, $|S| \le \boldsymbol{c^{k+3}}$. □

Similarly to Lemmas 10.2–10.3 and Theorems 10.5–10.7, using Lemma 10.8, all complexity bounds of NetDAG can be restated in terms of the expansion constant c_α. Specifically, the number of children per node becomes c_α^6 (applying Lemma 10.8 with $r = \alpha \cdot 2^i$ and $k = \lceil \log_2(3 + 2\tau/(\alpha \cdot 2^i)) \rceil \le 3$), giving a query time complexity of $O(c_\alpha^6 \cdot \phi)$ and a construction time complexity of $O(c_\alpha^{O(1)} N\phi)$. The space complexity remains $O(N)$. These bounds are summarized in Table 2.

10.1.2 Full derivations of the analysis of NetDAG *and other doubling-based indexes.* We derive the entries in Table 2. Throughout, r denotes r_i and the parent-layer radius is $2r$ ($\kappa = 2$).

NetDAG**.** We analyze the query time complexity of NetDAG in terms of both λ and c, for comparison with methods that use either measure of dimensionality. In terms of λ, the number of children is $\lambda_\alpha^{\log_2 20}$ (Lemma 10.2). In terms of c, the child range is $3r + 2\tau$ and the separation is r. By Lemma 10.8, the number of children is at most $c_\alpha^{\lceil \log_2(3+2\tau/r) \rceil + 3} \le c_\alpha^6$. Since Algorithm 1 maintains a single pivot (candidate set of size 1), the per-layer NDC equals the number of children: $\lambda_\alpha^{\log_2 20}$ (or c_α^6). At each layer, the pivot satisfies $d(p, Q) \le r + \tau$ (Theorem 4.2).

Cover Tree [4]. Cover Tree uses the expansion constant c. By the covering and separation properties (Section 2 of [4]), the child range is $2r$ and the child separation is r. The number of children is c^4 (Lemma 4.1 of [4]). The candidate set size is c^5 (Theorem 5 of [4]). The per-layer NDC is $c^4 \cdot c^5 = c^9$. Let a be the nearest neighbor of Q in $\mathcal{D}$, *i.e.,* $d(a, Q) \le \tau$. By induction on the covering property (Section 2 of [4]), the candidate set at each layer contains a point within $2r$ of a. By the triangle inequality, $d(\text{cand}, Q) \le d(\text{cand}, a) + d(a, Q) \le 2r + \tau$. Since all radii satisfy $r \ge \alpha$, c can be replaced by c_α.

Navigating Nets [30]. Navigating Nets uses the doubling constant λ. Let a be the nearest neighbor of Q in $\mathcal{D}$. The child range is $\gamma R = 8r$ ($\gamma = 4$, $R = 2r$; Section 2.1 of [30]) and the child separation is r (Lemma 2.1 of [30]). By Corollary 2, each node has at most $\lambda^{\log_2(4 \cdot 8r/r)} = \lambda^5$ children. For the candidate set, let Z_{2r} denote the candidate set maintained at the parent scale $R = 2r$. Lemma 2.4 of [30] gives $d(a, z) \le 2R = 4r$ for some $z \in Z_{2r}$, so $d(Q, Z_{2r}) \le d(Q, a) + 4r$. The filter retains candidates within $d(Q, Z_{2r}) + 2r$ of Q (Procedure Approx-NNS of [30]), giving a candidate range of at most $d(Q, a) + 6r \le 7r$ (since $d(Q, a) \le \tau \le \alpha \le r$). By Corollary 2 with separation r, the candidate set size is at most $\lambda^{\log_2(4 \cdot 7r/r)} = \lambda^{\log_2 28}$. The per-layer NDC is $\lambda^5 \cdot \lambda^{\log_2 28} = \lambda^{5+\log_2 28}$. At each layer, Lemma 2.4 of [30] at scale r gives $d(a, z) \le 2r$, so $d(z, Q) \le 2r + \tau$. Since all radii satisfy $r \ge \alpha$, λ can be replaced by λ_α.

Net-Tree [22]. Net-Tree uses the doubling constant λ. Let a be the nearest neighbor of Q in $\mathcal{D}$. The child range is $4r$ (Lemma 3.9(ii) of [22]) and the child separation is r (Lemma 3.9(iii) of [22]). By Corollary 2, each node has at most $\lambda^{\log_2(4 \cdot 4r/r)} = \lambda^{\log_2 16} = \lambda^4$ children. The candidate set has range $13r$ and separation r (Section 3.4 and Lemma 3.9(iii) of [22]). By Corollary 2, the candidate set size is at most $\lambda^{\log_2(4 \cdot 13r/r)} = \lambda^{\log_2 52}$. The per-layer NDC is $\lambda^4 \cdot \lambda^{\log_2 52} = \lambda^{4+\log_2 52}$. By the covering property (Lemma 3.9(iv) of [22]), $d(a, \text{cand}) \le 4r$. By the triangle inequality, $d(\text{cand}, Q) \le 4r + \tau$. Since all radii satisfy $r \ge \alpha$, λ can be replaced by λ_α.

10.1.3 Detailed construction of NetDAG*.* The construction of NetDAG is the incremental greedy permutation construction framework of Har-Peled and Mendel [22] with modifications. The greedy permutation iteratively generates the centers $p_1, p_2, \ldots$ and so on. Each new center p_k is the farthest unselected point from all previously chosen centers, so, the radii $r_1 \ge r_2 \ge \cdots$ form a non-increasing sequence. The algorithm maintains two per-center structures: (i) the set Friends$(\cdot)$ (the *friends list*), and (ii) the set ServedBy$(\cdot)$. At the kth iteration, the friends list of a center p_i contains all centers whose distance to p_i is at most $\min\{8r_k,\ 4r_i\}$, where $4r_i$ reflects the radius when p_i was first selected and $8r_k$ is the current trimming threshold. ServedBy(p_i) stores the points for which p_i is currently the nearest selected center.

To describe Algorithm 5, following [22], we define the term *phase*: A phase begins at iteration i and ends at the first $j > i$ with $r_{j-1} \le r_{i-1}/2$; that is, the radius halves once per phase.

- **Initialization.** An arbitrary point p_1 is chosen as the first center. For every other point $x \in \mathcal{D}$, Algorithm 5 computes $\delta(x) = d(x, p_1)$, where $\delta(x)$ always denotes the distance from x to its *current* nearest selected center. Each such point records its assigned center in Center(x), is inserted into ServedBy(p_1), and is pushed into the max-heap H keyed by $\delta(\cdot)$. This prepares the greedy selection of farthest-first centers.
- **Selecting new centers and determining the number of layers.** In every iteration, the algorithm extracts from H the farthest unselected point (Line 15), which becomes the next center p_k. $r_k = \delta(p_k)$ is exactly its distance to the nearest earlier center.

Algorithm 5: NetDAG Construction Algorithm

Input: $\mathcal{D}$: data points; $\alpha > 0$; $0 \le \tau \le \alpha$.
Output: Layers $Y_0, \ldots, Y_\phi$; edge set E (within NetDAG) and E' (between Y_1 and Y_0).

```
// Initialization (same as greedy permutation)
1  E ← ∅;
2  E' ← ∅;
3  choose any p_1 ∈ D as the first center;
4  mark p_1 as selected;
5  δ(p_1) ← 0;
6  H ← empty max-heap with key = δ(x);
7  foreach x ∈ D with x ≠ p_1 do
8      δ(x) ← d(x, p_1);
9      Center(x) ← p_1;
10     insert x into ServedBy(p_1);
11     push(x, H);
12 k ← 1;
13 r ← +∞;
14 while r > α do
       // Select new center and determine layers (same as greedy
          permutation)
15     u ← pop(H); k ← k + 1; p_k ← u; r ← δ(u);
16     mark p_k as selected;
17     if k = 2 then
18         φ ← ⌈log_2(r/α)⌉;
19         Y_φ ← {p_1};
20         j ← φ − 1;
       // Create layers and assign edges (our modification)
21     if k ≥ 2 and j ≥ 1 and r ≤ α · 2^j then
22         i_min ← max(1, ⌈log_2(r/α)⌉);
23         for i = j to i_min do
24             Y_i ← {p_1, ..., p_{k−1}};
25             foreach q ∈ Y_{i+1} do
26                 foreach p_ℓ ∈ Friends(q) with p_ℓ ∈ Y_i do
27                     if d(p_ℓ, q) ≤ 3α · 2^i + 2τ then
28                         add (q, p_ℓ) to E;
29         j ← i_min − 1;
       // Update friends and served points (same as greedy
          permutation)
30     update all affected Friends(·) and ServedBy(·) using Algorithm 6;
31 Y_0 ← {p_1, ..., p_{k−1}};
   // Connect Y_1 to Y_0 for EPF (our modification)
32 foreach q ∈ Y_1 do
33     foreach p_ℓ ∈ Friends(q) with p_ℓ ∈ Y_0 do
34         if d(p_ℓ, q) ≤ 3α + 2τ then
35             add (q, p_ℓ) to E';
36 return (Y_0, ..., Y_φ, E, E');
```

When $k = 2$, this value determines the number of layers:

$$\phi = \lceil \log_2(r_2/\alpha) \rceil,$$

and the top layer is initialized as $Y_\phi = \{p_1\}$ (Line 19), which is an $(\alpha \cdot 2^\phi)$-net.

- **Creating layers.** Since the radii satisfy $r_1 \ge r_2 \ge \cdots$, whenever $r_k \le \alpha \cdot 2^j$ for the first time, the layer Y_j should be created. If r_k crosses multiple thresholds at once, all layers from j down to

$$i_{\min} = \max(1, \lceil \log_2(r_k/\alpha) \rceil)$$

are created in sequence in Lines 23–28. Each layer Y_i is simply the prefix $\langle p_1, \ldots, p_{k-1} \rangle$, which, as a set $\{p_1, \ldots, p_{k-1}\}$, is an r_k-net by Definition 2.5.

- **Assigning parent–child edges.** For each newly created layer Y_i, the algorithm examines every $q \in Y_{i+1}$. Its potential parents are restricted to $\text{Friends}(q) \cap Y_i$. For each such p_ℓ that satisfies

$$d(p_\ell, q) \le 3\alpha \cdot 2^i + 2\tau,$$

Algorithm 6: Procedure for updating the global Friends(·) and ServedBy(·)

Input: new center p_k with radius r

```
1  let c'_k be the center that served p_k two phases earlier;
2  foreach y ∈ Friends(c'_k) do
3      if d(p_k, y) ≤ 4r then
4          add y to Friends(p_k);
5          add p_k to Friends(y);
6  foreach y ∈ Friends(p_k) do
7      foreach x ∈ ServedBy(y) with x not selected do
8          d_new ← d(x, p_k);
9          if d_new < δ(x) then
10             remove x from ServedBy(Center(x));
11             Center(x) ← p_k;
12             insert x into ServedBy(p_k);
13             δ(x) ← d_new;
14             update H with the new δ(x);
```

Algorithm 5 adds an edge (q, p_ℓ) (Line 28).

- **Updating friends lists and served points.** Line 30 of Algorithm 5 invokes Algorithm 6. This update procedure follows the friends-list maintenance strategy of Har-Peled and Mendel's net-tree construction [22]. Let c'_k be the center that served p_k two phases earlier (Line 1). Its friends list $\text{Friends}(c'_k)$ provides the complete set of candidate centers that may be inserted into $\text{Friends}(p_k)$ (Line 2). For each candidate y, if $d(p_k, y) \le 4r$, the pair is inserted into each other's friends list. Algorithm 6 then examines all served points in $\text{ServedBy}(y)$ for every $y \in \text{Friends}(p_k)$ (Lines 6–14). Whenever a point x satisfies $d(x, p_k) < \delta(x)$, the algorithm updates $\delta(x)$, reassigns x to $\text{ServedBy}(p_k)$, and updates H accordingly. To control the size of friends lists, any center whose distance exceeds the $8r_k$ threshold is removed during scanning.
- **Connection to EditPathForest.** When the first index t with $r_t \le \alpha$ is reached, the prefix $\{p_1, \ldots, p_{t-1}\}$ becomes the α-net Y_0. Although Y_0 is not part of the NetDAG, it is generated by the same greedy-permutation process. The leaf layer of NetDAG is Y_1, while Y_0 serves as the root set of the small-step index EditPathForest. Edges between Y_1 and Y_0 are created using the same rule $d(p, q) \le 3\alpha + 2\tau$ as between the upper layers, ensuring that query processing can proceed seamlessly from the large-step index NetDAG into EditPathForest.

10.2 Supplementary Analysis of EPF

This subsection establishes the space and query time complexity of EPF. Recall that non-database nodes store graphs produced along the GEP from the root to a data graph. We bound the size of individual non-database nodes (Lemmas 10.10–10.11) and the total node count (Lemma 10.12). To aggregate the per-node bounds into a whole-tree bound, we construct an injective mapping from non-database nodes to data nodes (Lemmas 10.13–10.14), which yields the total size of all graphs (Lemmas 10.15–10.16) and the total number of edit operations (Lemma 10.17). These together give the space complexity of EPT (Corollary 3). Finally, we give an upper bound on the number of EPTs that should be traversed during a query (Lemma 10.18).

Foremost, we present/recall the terminologies used in the analysis. Each node of an EPT stores a graph. A *data node* is a node whose graph (called a *data graph*) belongs to the graph database $\mathcal{D}$.

A *non-database node* is a node whose graph does not belong to $\mathcal{D}$; such a graph (called a *non-database graph*) is produced along the GEP from G_{root} to some data graph.

Lemma 10.9 (Edge-edit Bound Between Two Graphs). *Given two graphs G_1 and G_2, there exists an edit sequence from G_1 to G_2 whose* edge-edit operations *are at most $|E(G_1)| + |E(G_2)|$.*

By the definition of GEP in Section 2.1, a GEP is the shortest edit sequence transforming G_1 into G_2. There exists a trivial edit sequence that deletes all edges of G_1 and then inserts all edges of G_2, which performs exactly $|E(G_1)| + |E(G_2)|$ edge-edit operations.

Lemma 10.10 (Upper Bound of the Size of Non-database Nodes in EPT). *Given an* EPT *rooted at G_{root} and a data graph set C, let $G \in C$ and let g be a non-database node that represents a non-database graph produced by the GEP from G_{root} to G. Then,*

$$|E(g)| \leq |E(G_{\text{root}})| + |E(G)|.$$

Proof. Since g appears as a non-database graph on the GEP from G_{root} to G, transforming G_{root} into g requires at least $|E(g)| - |E(G_{\text{root}})|$ edge insertions, and transforming g into G requires at least $|E(g)| - |E(G)|$ edge deletions. Thus the GEP performs at least $(|E(g)|-|E(G_{\text{root}})|)+(|E(g)|-|E(G)|) = 2|E(g)|-|E(G_{\text{root}})|-|E(G)|$.

By Lemma 10.9, the GEP from G_{root} to G performs at most $|E(G_{\text{root}})| + |E(G)|$ operations. If $|E(g)|$ exceeded this amount, the above lower bound would contradict Lemma 10.9. Therefore,

$$|E(g)| \leq |E(G_{\text{root}})| + |E(G)|.$$

□

Lemma 10.11 (Upper Bound of the Vertex Count of Non-database Nodes in EPT). *Given an* EPT *rooted at G_{root} and a data graph set C, let $G \in C$ and let g be a non-database node that is a non-database graph produced by the GEP from G_{root} to G. Then*

$$|V(g)| \leq \max\{|V(G_{\text{root}})|,\ |V(G)|\}.$$

Proof. As shown in [6], a GEP between two graphs cannot contain both vertex insertion and vertex deletion operations. Hence, along the GEP from G_{root} to G, the vertex count changes monotonically. Therefore, every non-database graph on this GEP, including g, satisfies

$$|V(g)| \leq \max\{|V(G_{\text{root}})|,\ |V(G)|\}.$$

□

Lemma 10.12 (EPF Node Count). *The total number of nodes in* EPF *is $O(N)$, where N is the number of graphs in the database.*

Proof. The total number of nodes in EPF equals the sum of the nodes across all its EPTs. Consider the i-th EPT that covers N_i data graphs, where K is the number of EPTs in EPF, $i = 1, \ldots, K$, and $\sum_i N_i = N$. Let m_i denote the number of leaf nodes in this EPT; since every leaf corresponds to a data graph assigned to this EPT, we have $m_i \leq N_i$. According to Definition 5.1, any non-database node must have multiple children. In a rooted tree, where every non-database internal node has out-degree at least 2, the number of such non-database nodes is bounded by the number of leaves, hence, at most m_i. Therefore, the total number of nodes in the i-th EPT is at most $N_i + m_i \leq 2N_i$. Summing over all trees in EPF gives $\sum_i O(N_i) = O(\sum_i N_i) = O(N)$. □

Algorithm 7: Construction of injective mapping π

Input: EPT with non-database node set I and data node set C
Output: Injective mapping $\pi : I \to C$

1 $U \leftarrow C$; // U: unassigned data nodes
2 **foreach** $g \in I$ *in order of decreasing depth* **do**
3 Let $\mathcal{T}_g$ be the subtree rooted at g;
4 Choose any $d \in \mathcal{T}_g \cap U$;
5 $\pi(g) \leftarrow d$;
6 $U \leftarrow U \setminus \{d\}$;
7 **return** π;

Lemma 10.13 (Number of Nodes in a Subtree of EPT). *Given an* EPT *with a data graph set C, consider any node u and the subtree $\mathcal{T}_u$ rooted at u. Let D_u be the set of nodes in $\mathcal{T}_u$ whose graphs are in C, and let I_u be the set of non-database internal nodes. Then*

$$|D_u| \ \geq\ |I_u|.$$

Proof. Consider an arbitrary node u and its subtree $\mathcal{T}_u$. By Definition 5.1 and the compression (Line 4 of Algorithm 2), every non-database node in $\mathcal{T}_u$ has out-degree at least 2, and every leaf in $\mathcal{T}_u$ represents a data graph; let L_u denote the set of leaves in $\mathcal{T}_u$. Hence, $L_u \subseteq D_u$.

Similar to Lemma 10.12, the number of non-database internal nodes in $\mathcal{T}_u$ is at most the number of leaves:

$$|I_u| \ \leq\ |L_u|.$$

Since $L_u \subseteq D_u$, we obtain $|L_u| \leq |D_u|$, and thus

$$|I_u| \ \leq\ |L_u| \ \leq\ |D_u|.$$

□

Lemma 10.14 (Assignment of Data Nodes to Non-database Nodes). *Given an* EPT *with data graph set C, let I be the set of non-database nodes in this* EPT*. Then, there exists an injective mapping*

$$\pi : I \to C$$

such that $\pi(g)$ is in the subtree rooted at g for every $g \in I$.

Proof. We first give a construction of π in Algorithm 7, and then show that it is injective and satisfies $\pi(g) \in \mathcal{T}_g \cap C$ for all $g \in I$.

Construction. We construct π as follows (Algorithm 7). We process all non-database nodes in I in order of decreasing depth. When processing a non-database node g, we choose any data node in its subtree $\mathcal{T}_g$ that has not been assigned to any previously processed non-database node, and set $\pi(g)$ to be this data node.

Correctness. We show that Line 4 always finds an available data node for every non-database node g.

Let C_g and I_g be the sets of data nodes and non-database nodes in $\mathcal{T}_g$, respectively. By Lemma 10.13, we have $|C_g| \geq |I_g|$.

When g is processed (Line 2), all previously processed non-database nodes in $\mathcal{T}_g$ are descendants of g (since we process by decreasing depth), so at most $|I_g| - 1$ of them have been assigned data nodes (Line 5) from C_g. Since $|C_g| \geq |I_g|$, at least one data node in C_g remains available for Line 4.

Since each data node is assigned at most once (Line 5), π is injective. By Line 4, $\pi(g) \in \mathcal{T}_g \cap C = C_g$ for all $g \in I$. □

Lemma 10.15 (Total Size of Non-database Graphs in EPT). *Given an* EPT *rooted at G_{root} with data graph set C, let I be the set of non-database nodes in this* EPT*. Then, the total size of all non-data*

graphs satisfies

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) = O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Proof. By Lemma 10.14, there exists an injective mapping $\pi : I \to C$, such that for every $g \in I$, the data node $\pi(g)$ is in the subtree rooted at g.

Consider an arbitrary $g \in I$. Since $\pi(g)$ is in the subtree rooted at g, the path from G_{root} to $\pi(g)$ in EPT passes through g. Thus, g is a non-database graph on the GEP from G_{root} to $\pi(g)$. By Lemma 10.10, $|E(g)| \le |E(G_{\text{root}})| + |E(\pi(g))|$. By Lemma 10.11, $|V(g)| \le \max\{|V(G_{\text{root}})|,\ |V(\pi(g))|\} \le |V(G_{\text{root}})| + |V(\pi(g))|$. Therefore,

$$|V(g)|+|E(g)| \le \big(|V(G_{\text{root}})|+|E(G_{\text{root}})|\big)+\big(|V(\pi(g))|+|E(\pi(g))|\big).$$

Summing over all $g \in I$ gives

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) \le |I|\cdot\big(|V(G_{\text{root}})|+|E(G_{\text{root}})|\big) + \sum_{g\in I}\big(|V(\pi(g))|+|E(\pi(g))|\big).$$

Since π is injective and $|I| \le |C|$,

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) \le |C|\cdot\big(|V(G_{\text{root}})|+|E(G_{\text{root}})|\big) + \sum_{G\in C}\big(|V(G)|+|E(G)|\big).$$

Since $|V(G)| = O(|E(G)|)$ for every graph G,

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) = O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

□

Lemma 10.16 (Total Size of All Graphs in EPT). *Given an* EPT *rooted at* G_{root} *with data graph set* C*, let* I *be the set of non-database nodes in this* EPT*. Then, the total size of all graphs in* EPT *is*

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Proof. The total size of all graphs in EPT is

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) + \sum_{G\in C}\big(|V(G)|+|E(G)|\big).$$

By Lemma 10.15,

$$\sum_{g\in I}\big(|V(g)|+|E(g)|\big) = O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Moreover, for each data graph $G \in C$ we have $|V(G)| = O(|E(G)|)$, and thus

$$\sum_{G\in C}\big(|V(G)|+|E(G)|\big) = O\Big(\sum_{G\in C}|E(G)|\Big).$$

Therefore, the total size of all graphs in EPT is

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

□

Lemma 10.17 (Total Number of Edit Operations on Edges in EPT). *Given an* EPT *rooted at* G_{root} *with a data graph set* C*, let* $\mathcal{E}$ *be the set of edges in this* EPT*. For each edge* $e \in \mathcal{E}$*, let* $\text{op}(e)$ *denote the number of edit operations represented by* e*. Then, the total number of edit operations represented by all edges in* EPT *satisfies the following*

$$\sum_{e\in\mathcal{E}}\text{op}(e) = O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Proof. For each data graph $G \in C$, the path from G_{root} to G in EPT represents a graph edit path (GEP) by Definition 5.1. Let $\text{ops}(G)$ be the total number of edit operations represented by the edges on this path.

To derive the upper bound of $\text{ops}(G)$, we consider the trivial edit sequence that transforms G_{root} into G by deleting all vertices and edges of G_{root} and then inserting all vertices and edges of G. This requires

$$|V(G_{\text{root}})|+|E(G_{\text{root}})|+|V(G)|+|E(G)| = O(|E(G_{\text{root}})|+|E(G)|)\,,$$

Since a GEP cannot be longer than this trivial sequence, we obtain

$$\text{ops}(G) = O(|E(G_{\text{root}})|+|E(G)|)\,.$$

Each data graph $G \in C$ has a root-to-G path with $\text{ops}(G)$ edit operations. The EPT is formed by merging shared prefixes of these paths into a tree. So, each tree edge appears on one or more such paths. Therefore, the total number of edit operations in the tree is at most the sum over all paths:

$$\begin{aligned}\sum_{e\in\mathcal{E}}\text{op}(e) &\le \sum_{G\in C}\text{ops}(G)\\ &= O\Big(\sum_{G\in C}\big(|E(G_{\text{root}})|+|E(G)|\big)\Big)\\ &= O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).\end{aligned}$$

□

Corollary 3 (Space Complexity of EPT)**.** Given an EPT rooted at G_{root} with data graph set C, let I be the set of non-database nodes and $\mathcal{E}$ be the set of edges, both in this EPT. Then the total space of EPT, including all graphs stored at nodes and all edit-operation sequences represented by its edges, is

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Proof. By Lemma 10.16, the total size of all graphs stored at the nodes in EPT is

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big),$$

and by Lemma 10.17, the total number of edit operations represented by all edges in EPT is:

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

Therefore, the total space of EPT is:

$$O\big(|C|\cdot|E(G_{\text{root}})| + \sum_{G\in C}|E(G)|\big).$$

□

Finally, we give an upper bound on the number of EPTs traversed during a query.

Lemma 10.18 (Upper bound on the number of EPTs to traverse). *Consider a graph similarity search query* (Q,τ) *in* Gisma*, where each* EPT *is built for a ball* $B(p,\alpha)$ *of radius* α *centered at a leaf pivot* p *(thus the set of* EPT *roots forms an* α*-net). Then, the number of* EPTs *that may contain answers (hence need to be traversed) is at most*

$$\lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}.$$

Proof. Fix a query (Q, τ). By construction, an EPT rooted at p indexes the data graphs inside $B(p, \alpha)$. If this EPT contains at least one answer, then there exists a graph g with $d(g, Q) \leq \tau$ and $d(g, p) \leq \alpha$. By the triangle inequality,

$$d(Q, p) \;\leq\; d(Q, g) + d(g, p) \;\leq\; \tau + \alpha \;=\; \alpha + \tau.$$

Hence, any EPT that contains an answer must have its root within the ball $B(Q, \alpha + \tau)$.

At the leaf layer, the roots of EPTs form an α-net; in particular, they are α-separated. Therefore, the set of candidate roots $Y' = \{\, p : \; d(Q, p) \leq \alpha + \tau \,\}$ is an α-separated subset contained in a ball of radius $R = \alpha + \tau$. Applying Corollary 2 with $r = \alpha$ and $R = \alpha + \tau$ yields

$$|Y'| \;\leq\; \lambda_\alpha^{\log_2\left(\frac{4R}{r}\right)} \;=\; \lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}.$$

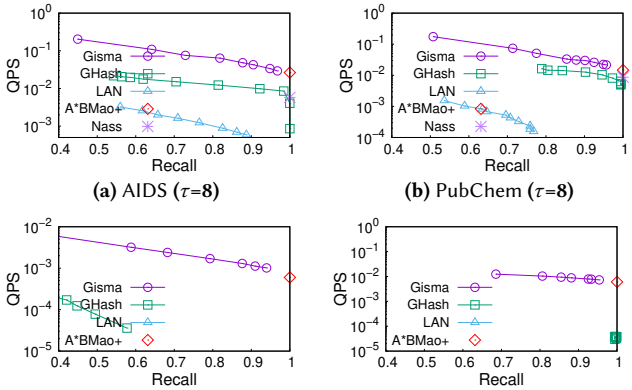
Hence, at most $\lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}$ EPTs need to be traversed. □

Query time complexity. As analyzed in Section 5.1, each EPT is queried via a DFS traversal visiting each node at most once (Algorithm 3), and the number of nodes in a EPT rooted at p is $O(|C_p|)$ by Lemma 10.12. By Lemma 10.18, the number of EPTs that need to be traversed is at most $\lambda_\alpha^{\log_2(4(1+\tau/\alpha))}$. Therefore, the total query time of EPF is $O(\sum_j |C_{p_j}|)$, where the sum is over the searched EPTs.

10.3 Supplementary Experimental Results

Supplementary table for EXP-1. Table 7 reports the per-query runtime, recall, and precision of the GED-based methods (GEDGW+, GEDIOT+, GEDHOT+, GHashing, and LAN) under default settings with varying τ. Recall is macro-averaged over queries that have at least one ground-truth answer, and precision is macro-averaged over queries that return at least one result.

Supplementary figure for EXP-2. Figure 13 presents the QPS-recall curves with a logarithmic QPS axis, complementing Figure 8 in the main text. The log scale better reveals the relative performance differences among methods with vastly different QPS values, such as exact methods (AStar-BMao+, Nass) and approximate methods.



(a) AIDS (τ=8) **(b)** PubChem (τ=8)

(c) Chemical1M (τ=8) **(d)** SYN (τ=4)

Figure 13: QPS-recall curves (log scale)

Table 7: Runtime comparison of GEDGW+, GEDIOT+, GEDHOT+, GHashing, **and** LAN **under default settings with varying** τ**. Format: seconds per query (Recall%/Precision%).**

Dataset	Method	Seconds per Query (Recall%/Precision%)					
		τ=2	4	6	8	10	12
AIDS	GEDGW+	0.4831 (18.9/100)	5.218 (19.8/100)	35.56 (14.7/100)	82.03 (12.3/100)	133.4 (12.1/100)	195.5 (14.0/100)
	GEDIOT+	0.1473 (2.8/60.0)	1.548 (26.2/32.4)	9.795 (31.3/15.5)	26.35 (40.1/17.3)	45.13 (49.3/26.3)	65.91 (55.1/37.5)
	GEDHOT+	0.6304 (21.7/92.0)	6.766 (41.7/43.3)	45.36 (40.4/19.1)	108.4 (46.0/19.4)	178.6 (53.2/27.8)	261.4 (58.4/38.9)
	GHashing	0.3861 (78.8/100)	0.5626 (70.6/100)	4.257 (64.0/100)	48.66 (56.3/100)	401.6 (48.6/100)	1885 (41.9/98.9)
	LAN	69.77 (67.4/100)	105.9 (71.6/100)	206.8 (81.2/100)	923.2 (82.4/100)	2906 (69.3/100)	CNF
		τ=2	4	6	8	10	12
PubChem	GEDGW+	0.3434 (0.7/100)	2.021 (2.4/100)	17.38 (2.2/100)	44.52 (2.1/100)	83.62 (2.0/100)	128.1 (2.1/98.3)
	GEDIOT+	0.0639 (13.9/23.3)	0.4286 (67.1/22.2)	3.245 (85.1/7.6)	10.70 (86.1/4.3)	22.87 (83.9/3.0)	36.28 (77.3/2.6)
	GEDHOT+	0.4073 (14.6/24.1)	2.450 (68.3/22.5)	20.63 (85.1/7.6)	55.22 (86.1/4.3)	106.5 (84.0/3.0)	164.4 (77.4/2.6)
	GHashing	0.2502 (96.4/100)	0.5605 (94.2/100)	4.978 (92.5/100)	68.57 (80.5/100)	821.0 (81.9/99.6)	5585 (75.3/98.6)
	LAN	326.1 (36.5/100)	486.0 (54.0/100)	664.3 (55.6/100)	962.7 (59.3/100)	1486 (49.1/98.8)	2345 (32.9/99.2)
		τ=2	4	6	8	10	-
Chemical1M	GEDGW+	23.52 (16.2/100)	222.1 (14.2/100)	761.5 (9.2/100)	1548 (8.8/100)	2393 (10.6/100)	-
	GEDIOT+	7.090 (13.4/39.7)	67.82 (37.8/6.2)	231.9 (46.0/5.2)	474.7 (58.1/9.9)	739.0 (66.7/18.3)	-
	GEDHOT+	30.61 (28.3/58.1)	289.9 (42.7/7.0)	993.4 (49.7/5.7)	2023 (61.0/10.4)	3132 (69.3/18.9)	-
	GHashing	7.797 (61.8/100)	13.04 (54.9/100)	126.5 (49.9/100)	5837 (42.1/100)	CNF	-
	LAN	76.79 (30.4/100)	655.4 (41.8/100)	CNF	CNF	CNF	-
		τ=1	2	3	4	5	6
SYN	GEDGW+	50.81 (43.1/100)	248.2 (38.5/100)	1283 (32.2/100)	2474 (25.7/100)	3852 (19.0/100)	5190 (13.8/100)
	GEDIOT+	18.42 (0.0/0.0)	90.15 (0.4/91.5)	479.6 (2.4/64.8)	957.1 (9.0/58.1)	1486 (14.0/44.7)	2003 (19.5/41.5)
	GEDHOT+	69.23 (43.1/100)	338.3 (38.7/99.9)	1763 (33.2/96.2)	3431 (30.2/82.4)	5337 (27.6/61.4)	7192 (27.7/50.2)
	GHashing	433.7 (100/100)	1654 (98.5/100)	7152 (97.2/100)	35141 (99.9/100)	87296 (99.6/100)	250898 (98.7/100)
	LAN	31.86 (9.0/100)	65.23 (2.7/100)	850.6 (16.5/100)	CNF	CNF	CNF

CNF: Cannot finish in 24h.